

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	4
1 Аналитическая часть	5
1.1 Формализация задачи коммивояжёра	5
1.2 Классический муравьиный алгоритм	6
1.2.1 Этап инициализации	6
1.2.2 Основной этап	7
1.2.3 Критерии остановки	8
1.2.4 Формализация муравьиного алгоритма	9
1.3 Модификации муравьиного алгоритма	10
1.3.1 Расширения муравьиного алгоритма	11
1.3.2 Гибридные алгоритмы	13
1.3.3 Алгоритмы с динамической параметризацией	17
1.4 Сравнение рассмотренных методов	22
2 Конструкторская часть	26
2.1 Обобщённый генетический алгоритм	26
2.1.1 Понятия	26
2.1.2 Схема генетического алгоритма	27
2.2 Описание предлагаемого метода	28
2.2.1 Требования к разрабатываемому методу	28
2.2.2 Формализация метода	29
2.3 Поиск муравьиным алгоритмом	31
2.3.1 Инициализация первого поколения	31
2.3.2 Формирование путей муравьиным алгоритмом	32
2.3.3 Обновление феромона	33
2.3.4 Выбор значений параметров	34
2.4 Формирование новой популяции генетическим алгоритмом	34
2.4.1 Алгоритмы селекции	35
2.4.2 Алгоритмы скрещивания	35
2.4.3 Алгоритмы мутации	37
2.5 Описание структур данных	38
2.5.1 Граф	38
2.5.2 Карта феромонов	38
2.5.3 Муравей	38

2.5.4	Популяция муравьёв	39
2.5.5	Итоговый выбор структур данных	40
2.6	Архитектурное разбиение на модули	40
2.6.1	Модуль структуры данных	41
2.6.2	Модуль ядро	41
2.6.3	Модуль стратегии	42
2.6.4	Модуль наблюдатели	42
3	Технологическая часть	44
3.1	Обоснования выбора программных средств реализации	44
3.1.1	Формат входных и выходных данные	44
3.1.2	Требования предъявляемые к ПО	44
3.1.3	Язык программирования	44
3.1.4	Интерфейс	45
3.2	Особенности реализации	45
3.2.1	Муравей	45
3.2.2	Локальная оптимизация	47
3.2.3	Колония	48
3.2.4	Стратегии	50
3.2.5	Наблюдатели	51
3.3	Тестирование	52
3.4	Пример работы ПО	52
4	Исследовательская часть	53
4.1	Исследование вариантов метода	53
4.1.1	Цель исследования	53
4.1.2	Описание исследования	53
4.1.3	Результаты исследования	55
4.2	Исследование сходимости метода	57
4.2.1	Цель исследования	57
4.2.2	Описание исследования	57
4.2.3	Результаты исследования	59
4.3	Исследование устойчивости метода	62
4.3.1	Цель исследования	62
4.3.2	Описание исследования	62
4.3.3	Результаты исследования	63
	СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	64
	Приложение Б	67

ВВЕДЕНИЕ

Задача коммивояжёра – одна из известных задач комбинаторной оптимизации, состоящей в оптимизации замкнутого маршрута по всем заданным точкам. Данная задача имеет длинную историю, впервые ей занялся Эйлер в 1736 году при рассмотрении задачи о семи Кенигсбергских мостах [4, с. 641]. Первым же неформальным упоминанием задачи считается книга 1832-го года «Коммивояжер – как он должен вести себя и что должен делать для того, чтобы доставлять товар и иметь успех в своих делах – советы старого курьера», где была описана проблема, но не дана формальная математическая модель для её решения, на основе которой, в последствие, У.Р. Гамильтон разработал математическую головоломку «Вокруг света» [1]. Первое формальное описание задачи коммивояжёра принадлежит Карлу Менгеру, который математически описал обобщённую задачу коммивояжёра [5].

В современном мире решения задачи коммивояжёра имеют существенное прикладное значение, в частности используя транспортную логистику для оптимизации маршрутов доставки товаров [2] или для планирования сложных путешествий через некоторое количество мест [3], оптимизация конвейера на заводе. Решение задачи коммивояжёра часто используется электронными магазинами, курьерскими службами, например одна из модификаций задачи коммивояжёра легла в основу соревнования E-CUP 2025: ML Challenge [6], что доказывает актуальность решения данной задачи.

Несмотря на длительную историю, до сих пор не найден алгоритм, для поиска точного решения задачи со сложностью не выше полиномиальной. В частности полный перебор имеет факториальную сложность. Однако существует множество алгоритмов, позволяющих найти не оптимальное решение, а приближённое, но за обозримое количество операций, например генетический алгоритм, метод имитации отжига, метод градиентного спуска, алгоритм искусственного роя и др. Одним из часто используемых известных эвристических методов является муравьиный алгоритм, однако у него есть ряд существенных проблем, таких как [7]:

- медленная сходимость;
- высокая вероятность падения в локальный минимум;
- подверженность к застою;
- сильная зависимость от входных данных и параметров;

Эти проблемы частично решаются с помощью модификаций классического алгоритма или комбинацией его с другими известными методами.

Целью данной работы является разработка модификации классического муравьиного алгоритма как метода решения задачи коммивояжёра.

1 Аналитическая часть

В аналитической части будет проведена формализация задачи коммивояжёра, описан классический муравьиный алгоритм и рассмотрены его основные модификации. Будет выполнен сравнительный анализ существующих подходов к решению задачи, выделены их достоинства и недостатки, а также обоснован выбор направления для разработки комбинированного метода.

1.1 Формализация задачи коммивояжёра

Словесно задача коммивояжёра формулируется следующим образом: коммивояжёр, выходящий из некоторого города желает посетить $n - 1$ других городов ровно по 1 разу и вернуться к исходному. Известно, что из каждого города можно перейти в любой другой и также известны расстояния между ними. Требуется установить в каком порядке коммивояжёр должен посещать города, чтобы пройденное им расстояние было минимальным [8].

Данное определение возможно формализовать математически, введя несколько определений из теории графов [9]:

Неориентированным графом (далее неорграф) G_n называется пара (V_n, E_n) , где V_n – конечное множество мощности n , элементы которого называются **вершинами**, а E_n – множество неупорядоченных пар $e_{ij} = \{v_i, v_j\}$ на V_n , элементы которого называют **рёбрами**

Неорграф G_n называется **полным**, если для любых пар вершин v_i, v_j ($i < j$) из множества V_n существует ребро e_{ij} в множестве E_n .

Конечной цепью X_k в неорграфе G_n называется произвольная упорядоченная последовательность вершин $v_{i_1}, v_{i_2}, \dots, v_{i_k}$ из множества V_n , таких, что $\forall j, 1 \leq j < k, e_{i_j, i_{j+1}} = \{v_{i_j}, v_{i_{j+1}}\} \in E_n$, т. е. такие, что соседние вершины пути связаны ребром.

Циклом Y в неорграфе G_n называют такую цепь, у которой первая и последняя вершина совпадают, при этом цикл называют **гамильтоновым**, если вершины, кроме первой и последней, не повторяются. В дальнейшем гамильтонов цикл будет записываться как $(v_{i_1}, v_{i_2}, \dots, v_{i_n})$, где v_{i_1} – первая и последняя вершина цепи.

Взвешенным неорграфом (размеченным) называется пара (G_n, F) , где F – функция сопоставляющая каждому ребру графа некоторое действительное положительное число, то есть $F : E_n \rightarrow \mathbb{R}$, называемая функцией разметки. При этом такое число называется **весом** или **длиной** ребра. Весом цепи X_k назовём сумму весов всех рёбер, входящих в него: $F(X_m) = \sum_{i=1}^{k-1} F(e_{j_i j_{i+1}})$, а весом цикла $Y = F(X_m) = F(e_{j_m j_1}) + \sum_{i=1}^{k-1} F(e_{j_i j_{i+1}})$.

Тогда, используя вышеизложенную терминологию, задачу коммивояжёра возможно определить следующим образом: Пусть дан полный взвешенный неорграф G_n и функция разметки над этим графом F . Требуется найти гамильтонов цикл с наименьшим (наибольшим) весом (оптимальный), среди всех остальных гамильтоновых циклов данного графа.

На рисунке 1.1 представлена формализация задачи коммивояжёра в виде IDEF0-диаграммы.

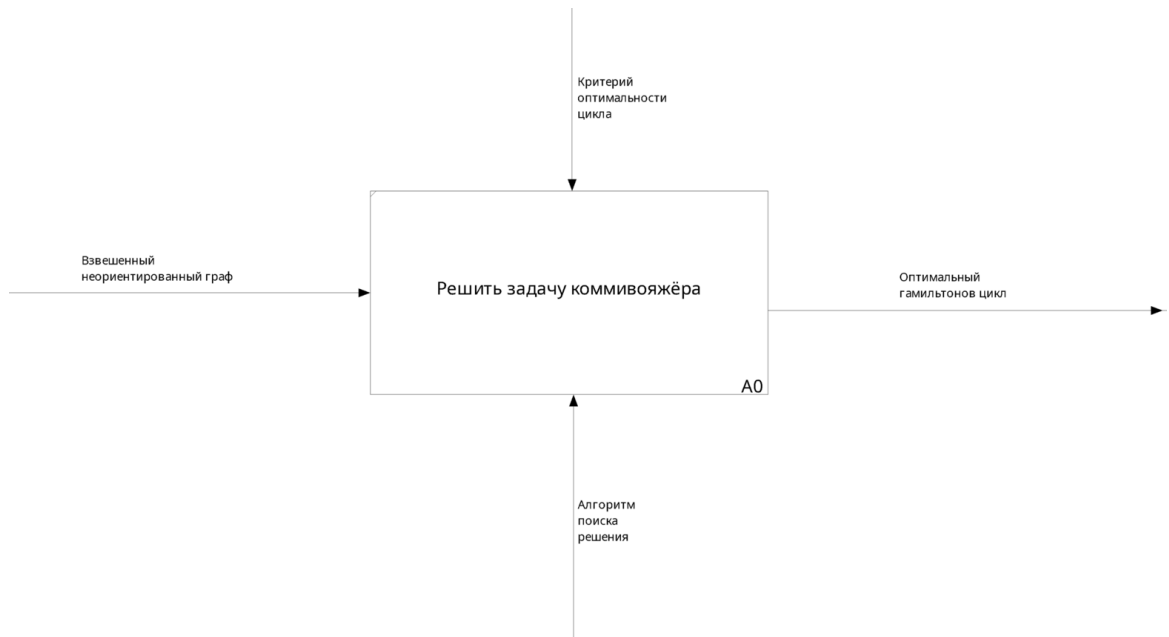


Рисунок 1.1 — Формализация задачи коммивояжёра в виде IDEF0-диаграммы

1.2 Классический муравьиный алгоритм

Классический муравьиный алгоритм впервые был предложен в [10] как новый подход к решению задач комбинаторной оптимизации, в частности, как решение задачи коммивояжёра. Метод построен на аналогии с тем, как функционируют муравьи при поиске маршрутов от колонии до источников пищи. Муравьи используют химические маркеры, феромоны, оставляемые на пройденном ими пути, как средство коммуникации. По началу муравьи ходят случайно, тем самым исследуя различные пути к источнику пищи. Пройдя путь в одну сторону и обратно муравьи оставляют феромоны на пройденном пути, причём концентрация оставленных ими на пути обратно зависит от длины или сложности пути. Последующие группы муравьёв смогут улавливать оставленный на путях феромон и с большей вероятностью будут выбирать те пути, на которых его больше, тем самым выбирая более простые и короткие пути.

Муравьиный алгоритм симулирует это поведение муравьёв на графе и итеративно ищет кратчайшие пути. На нулевой итерации происходит инициализация всей системы, а на последующих происходит поиск кратчайшего маршрута через все узлы, через имитацию муравьём с помощью упрощённых агентов.

1.2.1 Этап инициализации

Пусть дан взвешенный неограф (G_n, F) . Тогда $\tau_{ij}(k)$ – интенсивность феромона на ребре (v_i, v_j) графа G_n на k -ой итерации метода, где $1 \leq i, j \leq n$. Так как граф неориентированный, то $\tau_{ij}(k) = \tau_{ji}(k), \forall k$.

На нулевой итерации все ребра не обладают феромоном ($\tau_{ij}(0) = 0$), либо, как предложено в [10] обладают очень малым константным значением распылённого феромона ($\tau_{ij}(0) = c, c \geq 0$).

1.2.2 Основной этап

Основной этап описывает одну итерацию поиска пути оптимального пути. На каждой итерации запускается популяция искусственных агентов – **муравьёв**, каждый из которых независимо строит допустимый маршрут. Поведение агента определяется 3-мя его основными свойствами:

- **зрение** – агент способен оценить расстояние (вес) от одной вершины графа, до другой;
- **обоняние** – агент способен оценить концентрацию феромона, распылённого на каждом ребре графа, исходящим из вершины, где он находится;
- **память** – агент способен запоминать те вершины, на которых он уже был и, соответственно в дальнейшем посещать только не посещённые до этого вершины.

Построение маршрутов

Начиная с заданной или случайной вершины графа отдельный муравей итеративно выбирает по одному ребру, по которому перейти. Выбор ребра осуществляется с помощью стохастического правила, которое учитывает посещённые до этого муравьём вершины, распылённый на рёбрах феромон и их вес. Конкретное ребро для перехода определяется случайно, с вероятностями, описанными ниже.

Пусть на итерации k муравей a находится в вершине v_i , а множество ещё не посещённых им вершин обозначается как $U_a(i)$. Тогда вероятность выбора вершины $v_j \in U_a(i)$ в качестве следующей задаётся формулой:

$$P_{ij}^{(a)}(k) = \begin{cases} \frac{\tau_{ij}(k)^\alpha \cdot \eta_{ij}^\beta}{\sum_{v_l \in U_a(i)} \tau_{il}(k)^\alpha \cdot \eta_{il}^\beta}, & \text{если } v_j \in U_a(i); \\ 0, & \text{иначе,} \end{cases} \quad (1.1)$$

где

- $\tau_{ij}(k)$ – интенсивность феромона на ребре e_{ij} на итерации k ;
- η_{ij} – эвристическая значимость ребра;
- $\alpha \geq 0$ – параметр, определяющий значимость феромона при выборе пути;
- $\beta \geq 0$ – параметр, определяющий значимость эвристики при выборе пути.

Эвристическая значимость ребра η_{ij} описывает то, насколько ребро привлекательно для муравья без учёта распылённого на нём феромона. Эта оценка может быть задана любой функцией, главное её свойство – неизменность в течении всей симуляции. В [10] это функция определяется как обратно пропорциональная весу ребра: $\eta_{ij} = \frac{1}{F(e_{ij})}$.

Параметры α и β определяют относительную значимость феромонов и эвристики. При $\alpha = 0$ алгоритм становится полностью зависимым от эвристической функции, т. е. близким к жадному алгоритму. При $\beta = 0$ алгоритм не учитывает вес ребра, а значит становится полностью случайным (т. к. на начальной итерации концентрация феромона на всех рёбрах одинакова).

Каждый муравей завершает построение своего тура, когда множество $U_a(i)$ становится

пустым, после чего он возвращается в исходную вершину, таким образом формируя гамильтонов цикл.

После того как все муравьи завершили построение своих маршрутов, происходит вычисление длины каждого цикла:

$$L_a(k) = F(Y_a^k), \quad (1.2)$$

где Y_a^k – цикл, построенный муравьём a .

Минимальный среди всех $L_a(k)$ путь на итерации k называют **локально лучшим**.

Обновление феромонов

Заключительным этапом каждой итерации является обновление концентрации феромонов на каждом из рёбер. Этот процесс включается в себя два этапа.

Испарение феромона

Данный механизм действует как отрицательная обратная связь, призванный сделать менее удачные пути менее значимыми и предотвратить быстрого попадания алгоритма в локальный минимум. Новый уровень феромона для следующей итерации рассчитывается как:

$$\tau_{ij}(k+1) = (1-p)\tau_{ij}(k), \quad (1.3)$$

где $p \in [0, 1]$ – коэффициент испарения феромона.

Добавление нового феромона

Муравьи, построившие маршруты, добавляют феромоны на рёбра, по которым он прошли

$$\tau_{ij}(k+1) = \tau_{ij}(k) + \sum_{a=1}^m \Delta\tau_{ij}^{(a)}(k), \quad (1.4)$$

$$\Delta\tau_{ij}^{(a)}(k) = \begin{cases} \frac{Q}{L_a(k)}, & \text{если муравей } a \text{ прошёл через ребро } (i, j), \\ 0, & \text{иначе,} \end{cases} \quad (1.5)$$

где Q – положительная константа, определяющая масштаб увеличения концентрации феромона.

Поскольку увеличение концентрации обратно зависит от длины пути, то ребра, входящие в более короткие маршруты, получают больше феромона.

1.2.3 Критерии остановки

Итерационный процесс продолжается до выполнения одного из стандартных критериев:

- достижение заранее заданного числа итераций;
- отсутствие улучшений локального или глобального лучшего маршрута в течение некоторого количества итераций;
- достижение заданного порога качества решения.

На выходе алгоритма фиксируется лучший найденный гамильтонов цикл, который и

рассматривается как приближённое решение задачи коммивояжёра.

1.2.4 Формализация муравьиного алгоритма

На рисунках 1.2-1.3 представлена формализация муравьиного алгоритма в виде диаграмм в нотации IDEF0 1-го и 2-го уровня

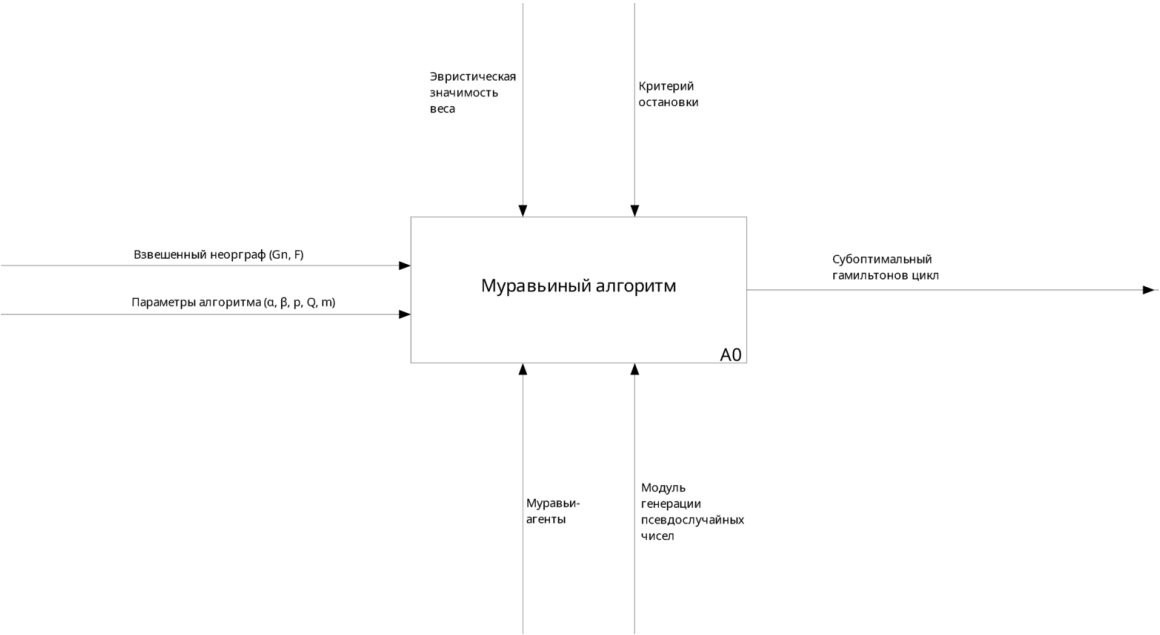


Рисунок 1.2 — Формализация муравьиного алгоритма в виде IDEF0-диаграммы 1-го уровня

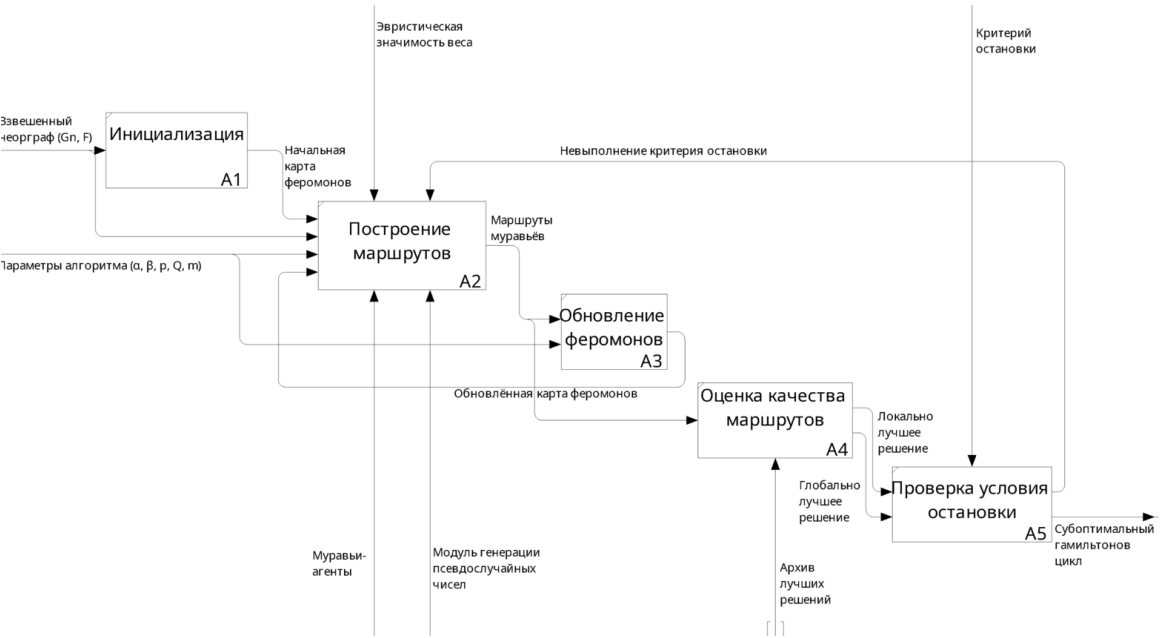


Рисунок 1.3 — Формализация муравьиного алгоритма в виде IDEF0-диаграммы 2-го уровня

1.3 Модификации муравьиного алгоритма

Как уже было указано во введении, классический муравьиный алгоритм обладает рядом существенных недостатков, обусловленных его структурой [7]:

— **Медленная сходимость.** На начальных итерациях феромон либо отсутствует, либо распределён равномерно, поэтому перемещение муравьёв почти случайно. Различия в концентрации феромона накапливаются медленно из-за испарения и случайности выбора переходов, что приводит к медленному формированию предпочтительных маршрутов.

— **Попадание в локальные минимумы.** Если несколько муравьёв случайно усилят подоптимальный маршрут, концентрация феромона на его рёбрах быстро возрастает, и остальные агенты начинают повторять этот путь. Из-за формулы вероятностного выбора возникает эффект самоусиления, приводящий к преждевременной фиксации не оптимального решения.

— **Склонность к застою.** При доминировании влияния феромона алгоритм концентрируется на одном маршруте, альтернативы быстро теряют феромон и перестают рассматриваться. В результате муравьи фактически перестают исследовать пространство решений, и процесс оптимизации останавливается.

— **Сильная зависимость от параметров.** Эффективность работы алгоритма существенно зависит от выбора параметров α , β , ρ и Q . Небольшие изменения их значений значительно смещают баланс между исследованием и использованием накопленной информации, что влияет на стабильность и качество результата.

— **Высокая вычислительная сложность.** На каждой итерации каждый муравей строит полный маршрут, что требует порядка $O(n^2)$ операций, а также производится обновление феромона на всех рёбрах графа $O(n^2)$. В совокупности это делает одну итерацию алгоритма достаточно трудоёмкой.

Модификации муравьиного алгоритма направлены на устранение описанных выше проблем и достигаются за счёт изменения одной или нескольких структурных частей классического метода:

— **стратегии выбора пути** – изменение правил выбора следующей вершины, использование локальных пост- и пред-оптимизаций, дополнительных ограничений;

— **правил обновления феромона** – модификация схем испарения и обновления феромона;

— **критериев остановки** – изменение условий остановки на учитывающие процесс поиска решения.

В рамках данной работы выделены три большие группы существующих модификаций муравьиного алгоритма:

1) **расширения классического муравьиного алгоритма** – модификации, изменяющие отдельные структурные компоненты базового метода;

2) **гибридные алгоритмы** – подходы, сочетающие муравьиный алгоритм с другими

эвристическими или метаэвристическими методами;

3) **алгоритмы с динамической параметризацией** – методы, в которых параметры α , β , ρ , Q и другие регулируются динамически в процессе работы.

Далее будет подробно рассмотрена каждая из групп.

1.3.1 Расширения муравьиного алгоритма

Расширения представляют собой модификации классического муравьиного алгоритма, которые сохраняют его базовую структуру, но изменяют отдельные стратегии, например выбор маршрута, обновления феромона или добавляют дополнительные управляющие конструкции.

Элитарный муравьиный алгоритм (ЭМА)

Идея элитарных муравьёв основана на концепте элитарности из других эволюционных алгоритмов, например генетическом алгоритме, где лучшие особи (представители) поколения или всех поколений сохраняются на протяжении длительного числа итераций или влияют на процесс поиска решений [24]. Ключевая идея метода в том, чтобы усилить влияние лучшего найденного маршрута на формирование феромонов, тем самым увеличивая вероятность того, что муравью будут включать участки лучшего пути в свои маршруты, что должно повышать сходимость алгоритма.

Модификация изменяет процесс изменения феромонов, добавляя элитных муравьёв, которые всегда ходят по лучшему найденному маршруту в текущей итерации или всех итерациях. Таким образом общая формула изменения феромонов становится:

$$\tau_{ij}(k+1) = (1 - \rho)\tau_{ij}(k) + \sum_{a=1}^m \Delta\tau_{ij}^{(a)} + e \cdot \Delta\tau_{ij}^{best}, \quad (1.6)$$

где

- e – количество элитных муравьёв;
- $\Delta\tau_{ij}^{best} = \frac{Q}{L_{best}}$, если ребро (i, j) принадлежит лучшему пути, иначе 0;
- L_{best} – вес лучшего найденного решения в поколении или за все поколения;

Улучшения относительно классического алгоритма

- ускоренная сходимость за счёт усиленной эксплуатации лучшего пути;
- уменьшение количества случайных отклонений после того, как найден хороший маршрут.

Ухудшения относительно классического алгоритма

- увеличенный риск падения в локальный минимум и стагнации.

Макс-мин муравьиный алгоритм (МММА)

Макс-мин муравьиный алгоритм, предложенный в [12] стремится улучшить два момента в классическом муравьином алгоритме:

- 1) увеличить сходимость, за счёт эффективного использования лучших найденных решений;
- 2) уменьшить вероятность падения в локальный минимум.

Эти улучшения достигаются за счёт ряда изменений, касающихся управления феромоном:

- 1) в каждой итерации только один муравей распыляет феромон на его пути. Это может быть как муравей, с лучшим путём в текущей итерации, так и муравей, с лучшим путём за все итерации:

$$\tau_{ij}(k+1) = (1 - \rho)\tau_{ij}(k) + \Delta\tau_{ij}^{best}$$

Распыление феромона только на одном, лучшем пути выделяет его на фоне остальных, что, как и в элитарном варианте, увеличивает шанс включения его участков в пути других муравьёв;

- 2) возможные значения концентрации феромона на всех рёбрах ограничено отрезком $[\tau_{min}, \tau_{max}]$, задаваемых как параметры:

$$\tau_{ij}(k+1) = \min(\tau_{max}, \max(\tau_{min}, \tau_{ij}(k+1)))$$

Введение ограничений на уровни феромона позволяют избежать стагнации и падения в локальный минимум, так как с одной стороны все рёбра будут иметь хоть какое-то количества феромона, что даёт шанс их выбора, с другой не даёт локально лучшим путям получить сверх высокие значения феромона;

- 3) начальное значение феромона на рёбрах устанавливается на τ_{max} , что увеличивает вероятность исследования в начале алгоритма.

Улучшения относительно классического алгоритма

- уменьшение риска падения в локальный минимум, за счёт ограничения феромона;
- ускорение сходимости, за счёт использования лучших путей;
- уменьшение операций распыления феромона.

Ухудшения относительно классического алгоритма

- увеличенная сложность параметризации, чувствительность к выбору границ.

Ранговый муравьиный алгоритм (РМА)

Ранговый муравьиный алгоритм [13] представляет собой дальнейшее развитие элитарного муравьиного алгоритма, в котором ещё больше взято от эволюционных алгоритмов, в частности генетического алгоритма. В отличие от классического муравьиного алгоритма, в данном подходе при распылении феромона участвуют не все муравьи, а только w лучших.

Муравьи упорядочиваются по возрастанию длины построенного пути:

$$L_1(k) \leq L_2(k) \leq \dots \leq L_m(k),$$

где $L_1(k)$ – лучший маршрут.

Каждый из первых w лучших муравьёв вносит вклад в феромон пропорционально своему рангу:

$$\Delta\tau_{ij}^{(r)}(k) = \begin{cases} \frac{w-r}{w} \frac{Q}{L_r(k)}, & \text{если муравей } r \text{ прошёл через ребро } (v_i, v_j), \\ 0, & \text{иначе,} \end{cases} \quad (1.7)$$

Также как и в элитарном варианте дополнительно добавляется феромон по лучшему пути, давай в итоге формулу:

$$\tau_{ij}(k+1) = (1-\rho)\tau_{ij}(k) + \sum_{r=1}^w \Delta\tau_{ij}^{(r)}(k) + \Delta\tau_{ij}^{best}, \quad (1.8)$$

Улучшения относительно классического алгоритма

- ускоренная сходимость за счёт использования лучших путей;
- уменьшения шанса падения в локальный минимум, за счёт нескольких лучших путей, покрывающих граф шире;

Ухудшения относительно классического алгоритма

- сложность подбора оптимального параметра w ;
- дополнительные вычисления при поиске лучших путей в каждом поколении;

1.3.2 Гибридные алгоритмы

Основной идее гибридных алгоритмов является комбинирование муравьиного алгоритма с другими эвристиками с целью закрытия их слабых мест друг другом. Механизмы их гибридизации могут различны, начиная с постобработки решений одного алгоритма другим, заканчивая ограничением выборов одного алгоритма другим.

Гибрид муравьиного алгоритма и табу-поиска (МАТП)

Гибридизация муравьиного алгоритма с табу-поиском [14] (TS) строится на идее сочетания глобального поиска муравьиного алгоритма и локального поиска за счёт использования механизма краткосрочной памяти, характерного для табу поиска.

Табу-поиск работает следующим образом:

1) начиная с некоторого заданного решения, генерируются соседние с ним решения из которых выбирается лучшее. В качестве соседних берутся полученные следующими операциями:

- 2-opt: удаление двух рёбер и пересоединение путей другим способом;
- 3-opt: удаление трёх рёбер и пересоединение;
- Swap (обмен): перестановка двух узлов в маршруте;

— Insertion: перемещение узлов в другую позицию.

2) в табу список заносится информация, обратная сделанному изменению. Таким образом запрещаются дальнейшие операции, которые приведут к уже исследованному решению;

3) для текущего решения подбираются соседние с ним, из которых выбирается лучшее, но которое не противоречит списку табу. Если существует хотя бы одно решение, не противоречащее списку табу, то выбирается лучшее из них, изменение заносится в список табу и алгоритм продолжается. Если не существует решений, непротиворечащих списку, то алгоритм заканчивается. На данном этапе возможно, что лучшее решение хуже текущего, тем не менее оно все равно будет использовано, чтобы избежать локального минимума.

Табу-поиск в данном гибридном варианте применяется ко всем путям, которые были построены муравьями, при этом феромоны обновляются уже по лучшим путям, полученным табу-поиском (если он стал лучше после табу-поиска).

Улучшения относительно классического алгоритма

- улучшение качества найденных решений;
- ускорение сходимости, за счёт детерминированного выбора более коротких участков;

Ухудшения относительно классического алгоритма

- увеличенная вычислительная сложность;

Гибрид муравьиного алгоритма и алгоритма слизевика (МААС)

В отличие от табу-поиска, где комбинация алгоритмов происходила за счёт локального улучшения решений построенных с помощью классического муравьиного алгоритма, данный вариант предлагает другую идею гибридизации алгоритмов.

Алгоритм слизевика (SMA) [15] основан на биологическом феномене: простейший организм «Physarum polycephalum» способен находить кратчайшие пути в лабиринтах из точки его локализации в точку с пищей, распространяя себя как сеть трубочек, которые утолщаются, если они эффективнее остальных передают пищу.

Для решения задачи коммивояжёра слизевик имитируется как динамическая трубопроводная сеть, в которой каждому ребру графа сопоставлены несколько переменных, описывающих его физические характеристики.

Основными переменными модели являются:

- L_{ij} – вес ребра e_{ij} ;
- $D_{ij}(t)$ – проводимость трубопровода на ребре e_{ij} в момент времени t ;
- P_i – «давление» в вершине v_i , формируемое в соответствии с законами сохранения потока;
- Q_{ij} – поток вещества по ребру e_{ij} , определяемый разностью давлений и проводимо-

стью.

Имитация слизевика происходит через итеративное моделирование физических процессов в трубах согласно законам гидродинамики. При этом используется следующая последовательность действий:

1) вычисляется давление во всех заданных узлах графа по закону Кирхгофа:

$$\sum_i \frac{D_{ij}}{L_{ij}} (P_i - P_j) = S_j,$$

где S_j равна положительному потоку источника, если j – начальная точка, отрицательному – если j – конечная точка и нулю в иных случаях.

2) вычисляется поток по каждому ребру, согласно формуле:

$$Q_{ij} = \frac{D_{ij}}{L_{ij}} (P_i - P_j).$$

3) рассчитывается проводимость всех рёбер в следующий момент времени:

$$D_{ij}(t+1) = D_{ij}(t) + \left(\frac{|Q_{ij}|}{1 + |Q_{ij}|} - D_{ij}(t) \right) \Delta t.$$

Итерации продолжаются до тех пор, пока значение D_{ij} не стабилизируется и изменения между шагами не станут меньше заданного порога. Лучшими узлами считаются те, которые имеют высокие показатели потока Q_{ij} и проводимости D_{ij} . Лучший путь при этом строится жадным алгоритмом начиная с начальной точки, что приводит к тому, что плохо учитывается завершающий цикл переход.

При гибридном подходе, предложенном в [15], в начале алгоритма единожды просчитывается путь слизевика и в список L_{AB} отбираются рёбра, которые имеют показатели потока Q_{ij} и проводимости D_{ij} выше заданного порога:

$$Q_{ij} > Q_{min}, D_{ij} > D_{min}$$

Далее используется классический муравьиный алгоритм, но с модифицированным правилом выбора очередного перехода на пути муравья:

- если среди возможных переходов есть тот, который есть в списке L_{AB} , то он выбирается безусловно;
- если такого перехода нет, то выбор проходит в соответствие с обычным вероятностным правилом муравьиного алгоритма.

Улучшения относительно классического алгоритма

- улучшение качества найденных решений;
- ускорение сходимости, за счёт детерминированного выбора качественных частей;

Ухудшения относительно классического алгоритма

- увеличение вычислительной сложности алгоритма, за счёт предрасчёта алгоритмом слизевика;
- увеличение вероятности падения в локальный минимум;
- существенное усложнение параметризации, из-за большого числа параметров.

Гибрид муравьиного алгоритма и генетического (ГАМА)

Генетический алгоритм (GA) [16] представляет собой метод оптимизации, основанный на принципах естественного отбора и наследования. Популяция возможных решений представляется в виде хромосом – обычных фиксированных решений задачи, т. е. в случае задачи коммивояжёра – циклов в графе. Алгоритм ищет решения скрещивая лучшие решения в популяциях. Процесс поиска организован циклически и включает в себя:

- 1) инициализация – формирование начальной популяции хромосом;
- 2) оценка приспособленности – вычисление качества каждого решения по заданной целевой функции;
- 3) отбор – выбор части особей для размножения пропорционально их приспособленности.
- 4) скрещивание – обмен фрагментами хромосом между выбранными родителями. В случае задачи коммивояжёра решения обмениваются участками их циклов, из которых получается новое решение;
- 5) мутация – случайные изменения генов, которое нужно для исследования новых решений. В случае задачи коммивояжёра это может быть обмен позициями в цикле двух узлов.

Процесс поиска, как и в муравьином алгоритме, проходит до определённого количества поколений или до найденного с заданной точностью решения.

Гибридный алгоритм ГАМА, предложенный в [16], комбинирует муравьиный и генетический алгоритмы, при этом в отличие от предыдущих гибридных алгоритмов, в данном варианте генетический алгоритм выступает в роли ограничителя для каждого из муравьёв.

Хромосома в данном случае не задаёт напрямую решение, а кодирует стратегию, по которой муравей должен выбирать следующий узел при построении пути. Хромосома представляет из себя непротиворечивые множество списков узлов, в которые муравей может отправиться на каждом шаге.

- 1) Создаётся первая популяция муравьёв со случайными хромосомами;
- 2) Муравьи строят маршруты с учётом ограничений из хромосомы:
 - если для текущего шага у муравья всего один разрешённый узел, то он выбирает его для перехода;
 - если таких узлов несколько, то выбор между ними делается по обычному вероятностному правилу муравьиного алгоритма.

3) Полученные решения оцениваются и из лучших происходит формирование следующего поколения хромосом за счёт скрещивания и мутации

4) Происходит обновление феромонов, как в классическом муравьином алгоритме.

Таким образом, в ГАМА комбинируются два вида обучения:

- усиление удачных рёбер муравьиным алгоритмом;
- улучшение стратегий выбора с сохранением разнообразия в маршрутах генетическим алгоритмом.

Улучшения относительно классического алгоритма

- уменьшение вероятности падения в локальный минимум, за счёт разнообразия в хромосомах;
- улучшение сходимости, за счёт двойного эволюционного подхода.

Ухудшения относительно классического алгоритма

- существенное увеличение вычислительной сложности, за счёт отбора лучших и скрещивания;
- усложнение параметризации, из-за большого числа параметров.

1.3.3 Алгоритмы с динамической параметризацией

В классическом муравьином алгоритме все параметры – α , β , p , m – фиксированы и задаются до начала алгоритма, однако в процессе проведения расчётов важность факторов может меняться, из-за чего константные параметры могут быть неэффективными, в определённые моменты поиска решения. Алгоритмы данного раздела вводят способы, как изменять параметры по ходу расчёта, для увеличения эффективности или решения проблем муравьиного алгоритма.

Адаптивный муравьиный алгоритм с динамическими коэффициентами (АМАДК)

В данном алгоритме, предложенном в [17] вводится ряд изменений относительно классического муравьиного алгоритма:

- вводятся формулы для динамического расчёта α и β для каждой итерации алгоритма;
- вводятся стратегии по локальным улучшениям полученных путей;
- изменяются способы обновления феромона на пройденных путях;

Формулы для динамического расчёта α и β

В данной модификации, в отличие от классического алгоритма, коэффициенты влияния феромона и эвристика рассчитываются динамически, для каждой итерации. При этом формулы их расчёта выбраны исходя из следующих тезисов: В начале алгоритма концентрация низка на всех рёбрах, поэтому выбор прежде всего строится на эвристике, и, чтобы избежать выбора только относительно эвристики, важно поддерживать высокое значение α с относительно низким β . Однако с продолжением расчётов и накоплением феромонов, дабы избежать паде-

ния в локальный минимум, но продолжать искать лучшие пути нужно постепенно уменьшать значения α и увеличивать β . При этом для увеличения разнообразия, добавляется случайный множитель. Предложенные формулы расчёта имеют вид:

$$\alpha(n_c) = \cos\left(\frac{r_1 n_c \pi}{2n_{c\max}}\right) + A,$$

$$\beta(n_c) = \sin\left(\frac{r_2 n_c \pi}{2n_{c\max}}\right) + B,$$

где

- $r_1, r_2 \in [0, 1]$ – случайные числа;
- $A = 2, B = 3$ – экспериментально подобранные константы;
- n_c – текущая итерация;
- $n_{c\max}$ – максимальное число итераций.

Адаптивный расчёт феромона

Данный алгоритм проводит обновление феромона в 2 этапа: локальный и глобальный.

Локально, концентрация феромона обновляется после каждого прохода муравья. Формула для этого обновления:

$$\tau_{ij}(t) = (1 - \varepsilon)\tau_{ij}(t) + \frac{\varepsilon}{m \cdot L_{nn}},$$

где

- $\varepsilon \in [0, 1]$ – коэффициент испарения, для локального обновления;
- L_{nn} – длина пути, полученного жадным алгоритмом.

Глобальное обновление происходит также, как и в ранговом муравьином алгоритме, однако коэффициент испарения ρ рассчитывается динамически для каждой итерации. По умолчанию он статичен и меняется в том случае, если лучшее решение не изменяется в течение нескольких итераций, что значит, что алгоритм попал в локальный минимум и коэффициент испарения нужно увеличить. Формула для изменения имеет следующий вид:

$$\rho(n_c + 1) = \begin{cases} \rho_0, & n_c < \omega \cdot n_{c\max} \\ \frac{\rho(n_c)}{\gamma}, & n_c > \omega \cdot n_{c\max} \text{ и } s > s_0, \end{cases}$$

где

- ρ_0 – начальное (максимальное) значение коэффициента испарения;
- $\omega \in [0, 1]$ – множитель, определяющий после какого числа операций требуется проверка на локальный минимум;
- s и s_0 – количество итераций без улучшения результата и пороговое значение соответственно;
- $\gamma \in [0, 1]$ – множитель коэффициента испарения.

Таким образом, данная модификация предлагает решения проблем стагнации и падения

в локальный минимум за счёт динамических манипуляций с параметрами, отвечающими за феромон и влияние феромона на путь муравьёв;

Улучшения относительно классического алгоритма

- улучшение качества результатов, за счёт использования более удачных параметров на разных этапах поиска;
- уменьшение вероятности падения в локальный минимум, за счёт эвристики в начале поиска и увеличение испарения при стагнации;

Ухудшения относительно классического алгоритма

- чувствительность к параметрам в формулах выбора пути;
- эмпиричность формул, которые могут быть не эффективными для разных классов задач;
- существенное усложнение параметризации, из-за большого числа параметров.

Муравьиный алгоритм с динамическим коэффициентом испарения и условием завершения (МАЭ)

Данный алгоритм, описанный в [18], вводит понятие энтропии решений муравьёв, на основе которых в нём динамически меняются параметры и проверяется условие завершения алгоритма. Относительно классического муравьиного алгоритма, в данном варианте введены 3 изменения:

- 1) кластеризация узлов;
- 2) адаптивное испарение феромонов;
- 3) условие завершения на основе энтропии.

В классическом муравьином алгоритме при построении маршрута муравей выбирает следующий узел из всех еще не посещенных узлов, что приводит к вычислительной сложности $O(n^2)$ для построения маршрута одного муравья. В МАЭ для каждого узла создается набор кластеров, которые вместе содержат все остальные узлы. Кластеры упорядочены по вероятности того, что узел в кластере является частью оптимального маршрута. Наиболее вероятные узлы помещаются в первичные кластеры.

Процесс построения маршрута муравья с использованием кластеризации узлов включает два этапа:

- 1) Выбор кластера: для последнего узла в текущем маршруте вычисляются вероятности выбора каждого кластера на основе средних значений эвристики и феромонов для узлов в кластере:

$$p(C_j) = \frac{(\eta_{C_j})^\alpha \cdot (\tau_{C_j})^\beta}{\sum_{i=1}^{n_{clusters}} (\eta_{C_i})^\alpha \cdot (\tau_{C_i})^\beta}$$

где η_{C_j} - средняя эвристическая информация (обратные расстояния) между текущим узлом и узлами кластера C_j , τ_{C_j} - средняя интенсивность феромона.

- 2) Выбор узла из кластера: если в выбранном кластере есть непосещенные узлы, то

следующий узел выбирается из них классическим способом. Если в выбранном кластере нет непосещенных узлов, то используются следующие кластеры по порядку.

Адаптивное управление феромонами на основе энтропии

Под энтропией в данной работе подразумевается степень разнообразия решений, полученных в рамках одного поколения. Для вычисления энтропии сначала определяется вероятность появления каждого ребра в решениях текущей популяции:

$$p_{ij} = \frac{m_{ij}}{m},$$

$$H = - \sum_{i=1}^n \sum_{j=i+1}^n p_{ij} \cdot \log_2 p_{ij},$$

где m_{ij} – количество муравьёв, прошедших через ребро e_{ij} .

Высокие значения энтропии соответствуют ситуации, когда муравьи используют разнообразные наборы рёбер, что характерно для начала алгоритма. Низкие значения энтропии указывают на концентрацию решений вокруг небольшого набора рёбер, что происходит при сходимости к локальному оптимуму. Относительно энтропии рассчитывается коэффициент испарения для каждого поколения:

$$\rho = \rho_{min} + (\rho_{max} - \rho_{min}) \cdot \frac{H - H_{min}}{H_{max} - H_{min}}$$

- при высокой энтропии $\rho \rightarrow \rho_{max}$, что ускоряет испарение и стимулирует исследование новых областей пространства поиска;
- при низкой энтропии $\rho \rightarrow \rho_{min}$, что замедляет испарение и позволяет углубить поиск более оптимального локального решения.

Условие завершения на основе энтропии

Традиционные критерии остановки (фиксированное число итераций или отсутствие улучшений) часто оказываются неоптимальными: либо преждевременно останавливают поиск, либо приводят к избыточным вычислениям.

Новое условие завершения использует энтропию как индикатор исчерпания потенциала улучшений:

$$H \cdot (1 + \omega) \leq H_{min}$$

где ω - малый положительный параметр, определяющий допустимую близость к минимальной энтропии.

Улучшения относительно классического алгоритма

- оптимизация числа итераций, за счёт динамического условия завершения;
- уменьшение вероятности падения в локальный минимум, так как коэффициент испарения увеличивается при низкой энтропии, что способствует разнообразию решения;

Ухудшения относительно классического алгоритма

- увеличение вычислительной сложности, за счёт просчёта энтропии на каждой итера-

ции;

— замедление сходимости, за счёт динамического испарения.

Муравьиный алгоритм с динамическими гетерогенными параметрами (ГДМА)

В данном алгоритме, предложенном в [19], реализуется идея гетерогенных, т. е. различных параметров муравьёв. Основная идея заключается в том, что каждый муравей в колонии обладает индивидуальными поведенческими характеристиками, которые эволюционируют в процессе поиска решения.

В отличие от классического подхода, где все муравьи используют одинаковые значения параметров α и β , в ГДМА каждый муравей k имеет собственные значения α_k и β_k , определяющие его предпочтения при выборе пути:

$$P_{ij}^k = \frac{[\tau_{ij}]^{\alpha_k} [\eta_{ij}]^{\beta_k}}{\sum_{l \in N^k} [\tau_{il}]^{\alpha_k} [\eta_{il}]^{\beta_k}}$$

где N^k - множество допустимых переходов для муравья k .

Параметры α и β у муравьёв адаптируются с использованием упрощённого эволюционного алгоритма, при этом процесс можно разделить на несколько итераций:

- 1) **инициализация:** по умолчанию все муравьи инициализируются некоторыми параметрами, при этом в самой [19] предложен вариант с жадными муравьями, где $\alpha_k = 0$ и $\beta_k = 10$;
- 2) **оценка эффективности:** каждые w итераций вычисляется средняя приспособленность каждого муравья на основе длины построенных им маршрутов.
- 3) **селекция:** выбирается муравей с наилучшей средней приспособленностью и муравей с наихудшей средней приспособленностью.
- 4) **мутация:** параметры лучшего муравья подвергаются гауссовской мутации для создания нового потомка:

$$\alpha_{child} = \alpha_{best} + \mathcal{N}(0, \sigma), \quad \beta_{child} = \beta_{best} + \mathcal{N}(0, \sigma)$$

где $\sigma = 0.05$ - стандартное отклонение.

- 5) **замена:** созданный потомок заменяет наихудшего муравья в популяции.

Улучшения относительно классического алгоритма

- уменьшения вероятности падения в локальный минимум, за счёт разнообразия параметров у муравьёв;
- улучшение качества решения, за счёт подстраивания параметров, под разные этапы поиска;
- упрощение параметризации, меньшая чувствительность к начальным значениям, за счёт подбора параметров динамически.

Ухудшения относительно классического алгоритма

- увеличенная чувствительность к количеству итераций, так как подбор оптимальных параметров происходит в зависимости от входного графа;
- замедленная сходимость, за счёт подбора параметров.

1.4 Сравнение рассмотренных методов

Как уже было указано ранее, модификации муравьиного алгоритма и сам алгоритм, состоят из 3-х структурных частей:

- стратегия выбора пути;
- правила обновления феромона;
- критерии остановки.

Каждая модификация изменяет один или несколько механизмов, что и описывает её ключевые отличия от муравьиного алгоритма. Также в модификация меняется количество параметров, что влияет на сложность предварительной настройки. Эти пункты позволяют кратко описать основные новшества, введённые модификацией.

Любая модификация алгоритма стремится решить один или несколько недостатков классического, при этом зачастую жертвуя чем-то другим, поэтому важно указывать, что именно улучшила модификация, и чем ради этого пришлось пожертвовать.

В таблице 1.1 представлены сформулированные критерии сравнения рассмотренных модификаций муравьиного алгоритма.

Таблица 1.1 — Критерии сравнения модификаций муравьиного алгоритма

№	Критерий	Описание
1	Число гиперпараметров	Количество настраиваемых параметров, которые необходимо задать до начала работы алгоритма.
2	Стратегия построения цикла	Принцип, по которому отдельный муравьёв строит полный маршрут (тур).
3	Стратегия обновления феромона	Правила изменения уровня феромона на рёбрах графа (испарение и добавление).
4	Критерий остановки	Условие, при выполнении которого работа алгоритма завершается.
5	Улучшения	Улучшения, привнесённые модификацией по сравнению с классическим муравьиным алгоритмом.
6	Ухудшения	Недостатки или компромиссы модификации по сравнению с классическим муравьиным алгоритмом.

В таблицах 1.2- 1.4 представлено сравнение модификаций по сформулированным критериям.

Таблица 1.2 — Сравнение расширений классического муравьиного алгоритма

№	ЭМА	МММА	РМА
1	6 ($\alpha, \beta, \rho, m, Q, e$)	7 (+ τ_{min}, τ_{max})	6 ($\alpha, \beta, \rho, m, Q, w$)
2	Стандартная	Стандартная	Стандартная
3	Все муравьи + e по лучшему пути	Только лучший муравей. $\tau_{ij} \in [\tau_{min}, \tau_{max}]$	w лучших (взвеш. по рангу) + лучший путь
4	Стандартные	Стандартные	Стандартные
5	— Ускоренная сходимость	— Ускоренная сходимость — Снижен риск лок. минимума	— Ускоренная сходимость — Снижен риск лок. минимума
6	— Риск стагнации/лок. мин.	— Сложная параметризация (τ_{min}, τ_{max})	— Сложный подбор w — Доп. вычисления (ранжирование)

Таблица 1.3 — Сравнение гибридных модификаций муравьиного алгоритма

№	МАТП	МААС	ГАМА
1	5 ($\alpha, \beta, \rho, m, Q$)	8 ($\alpha, \beta, \rho, m, Q, Q_{min}, D_{min}, S$)	7 ($\alpha, \beta, \rho, m, Q$, параметры ГА)
2	Стандартный + лок. оптимизация	Безусловный выбор рёбер из пути слизевика, иначе стандартный	Стандартный, но из множества, разрешённого хромосомой
3	По улучшенным путям	Стандартный	Стандартный
4	Стандартные	Стандартные	Стандартные
5	— Качество решений — Ускоренная сходимость	— Качество решений — Ускоренная сходимость	— Устойчивость к лок. мин. — Улучшенная сходимость
6	— Высокая выч. сложность	— Высокая выч. сложность — Риск лок. минимума — Сложная параметризация	— Высокая выч. сложность — Сложная параметризация

Таблица 1.4 — Сравнение динамических модификаций муравьиного алгоритма

№	АМАДК	МАЭ	ГДМА
1	10 $(\alpha, \beta, \rho, m, Q, A, B, \omega, \gamma, s_0)$	7 $(\alpha, \beta, \rho, m, Q, \rho_{min}, \rho_{max}, H_{min}, \omega)$	7 $(\alpha, \beta, \rho, m, Q, \sigma, w_{adapt})$
2	Стандартный + лок. улучшения	Кластеризация узлов → выбор внутри кластера	Стандартный, но с индивид. параметрами
3	Ранговое + испарение растёт при стагнации	Испарение повышается при низкой энтропии решений	Стандартный
4	Стандартный	Энтропия меньше заданного порога	Стандартный
5	— Качество решений — Устранение стагнации	— Оптим. число итераций — Устранение стагнации	— Устранение стагнации — Качество решений — Проще параметризация
6	— Эмпиричность формул — Сложная параметризация	— Замедленная сходимость — Выч. сложность (H , кластеры)	— Замедленная сходимость — Чувств. к числу итераций

Проведённый сравнительный анализ модификаций муравьиного алгоритма для задачи коммивояжёра показывает, что каждая группа модификаций предлагает свой компромисс между качеством решения, скоростью сходимости, вычислительной сложностью и трудоёмкостью настройки. Базовые расширения (ЭМА, МММА, РМА) фокусируются на совершенствовании механизма феромонов, что улучшает сходимость, но не решает фундаментальных проблем стагнации и чувствительности к параметрам. Гибридные подходы (ACO-TS, МААС, ГАМА), интегрирующие внешние эвристики, демонстрируют значительный потенциал в повышении качества решений, однако зачастую ценой существенного усложнения алгоритма и роста вычислительных затрат.

Наиболее перспективными представляются алгоритмы с динамической параметризацией, так как они позволяют уменьшить зависимость алгоритма от входных данных. Особенно выделяется эволюционный подход к реализации подбора параметров, так как он позволяет динамически менять параметры в зависимости от конфигурации графа и процесса поиска решения. При этом наличие индивидуальных параметров позволяет поддерживать разнообразие решения, направляя его в наиболее эффективное русло. Рассмотренные выше модификации изменяют параметры с помощью эмпирических формул или случайного перебора, не учитывая

динамическую среду поиска, что ограничивает их возможность адаптивно подстраиваться под нее. Динамический подбор параметров можно оптимизировать путём использования алгоритмов, адаптирующих поведение муравьёв в зависимости от качества их решений.

Таким образом, объединение муравьиного алгоритма с более сложными эволюционным алгоритмом, таким как генетический, имеет высокие шансы оказаться более эффективным других подходов в неспецифичных случаях. Одним из вариантов дальнейшего развития муравьиного алгоритма предлагается «Метод решения задачи коммивояжёра муравьиным алгоритм с гетерогенными динамическими параметрами на основе генетического алгоритма», формализация которого в нотации IDEF-0 представлена на рисунке 1.4.

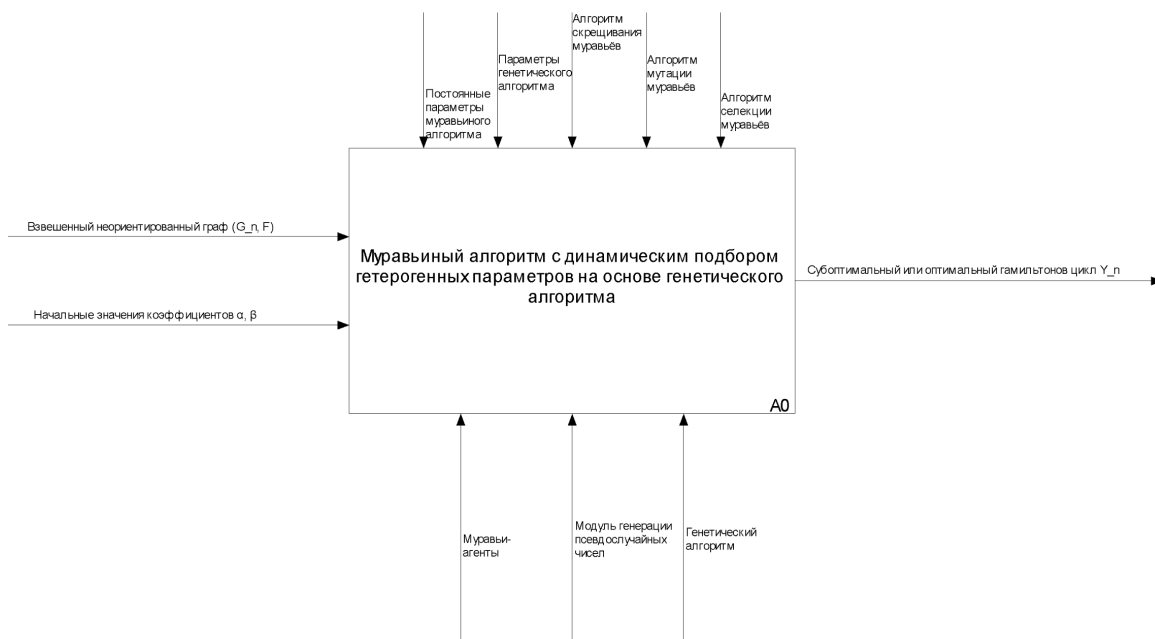


Рисунок 1.4 — Формализация предлагаемого метода в виде IDEF0-диаграммы

Вывод

В результате аналитической части была формализована задача коммивояжёра в терминах теории графов и нотации IDEF0, описан классический муравьиный алгоритм и выявлены его недостатки. Были рассмотрены и классифицированы существующие модификации муравьиного алгоритма, проведён их сравнительный анализ по ключевым критериям. На основе анализа обоснована перспективность использования эволюционного подхода для динамического подбора гетерогенных параметров, что позволило сформулировать предлагаемый метод решения задачи коммивояжёра муравьиным алгоритмом с гетерогенными динамическими параметрами на основе генетического алгоритма.

2 Конструкторская часть

В конструкторской части будет разработан комбинированный метод решения задачи коммивояжёра, основанный на муравьином алгоритме с динамическим подбором гетерогенных параметров с использованием генетического алгоритма. Будут описаны общая структура метода, детальные этапы его работы, используемые структуры данных, а также архитектурное разбиение на модули. Для наглядности основные алгоритмические решения будут представлены в виде IDEF0-диаграмм и схем алгоритмов.

2.1 Обобщённый генетический алгоритм

Для разработки предлагаемого комбинированного метода, необходимо рассмотреть основные принципы генетического алгоритма. Это обусловлено тем, что предлагаемый метод использует генетический алгоритм как механизм адаптации гетерогенных параметров муравьёв.

Генетический алгоритм представляет собой метаэвристику, вдохновлённую принципами естественного отбора и эволюции в живой природе. Впервые подобный подход был предложен Джоном Холландом в 1975 году [20]. Основная идея заключается в моделировании биологического процесса эволюции: популяция потенциальных решений задачи (особей) развивается из поколения в поколение за счёт применения операторов отбора, скрещивания и мутации. При этом более приспособленные особи, то есть дающие лучшее значение целевой функции, имеют больше шансов передать свои характеристики потомкам. В результате последовательных итераций популяция в целом улучшается, приближаясь к оптимальному решению.

2.1.1 Понятия

В рамках генетического алгоритма используются следующие базовые понятия [21]:

Особь – потенциальное решение задачи, представленное. Каждая особь обладает определённым значением приспособленности.

Хромосома – структура данных, кодирующая решение. Она состоит из генов, расположенных в определённом порядке. В зависимости от задачи гены могут быть бинарными, целочисленными, вещественными или иметь другую природу.

Ген – элементарная единица хромосомы, отвечающая за кодирование отдельного параметра решения. Совокупность генов полностью определяет характеристики особи.

Аллель – конкретное значение гена.

Популяция – множество особей, существующих в рамках одного поколения. Размер популяции является фиксированным параметром и определяет количество одновременно рассматриваемых решений.

Поколение – одна итерация эволюционного процесса, в результате которой из текущей популяции формируется новая.

Фитнес-функция (функция приспособленности) – критерий качества особи, который оценивает, насколько хорошо данное решение удовлетворяет поставленной задаче. Значение фитнес-функции определяет вероятность участия особи в размножении.

Операторы селекции, скрещивания, мутации – операции применяемые к хромосомам для получения и/или изменения особей.

2.1.2 Схема генетического алгоритма

Схема классического генетического алгоритма представлена на рисунке 2.1.



Рисунок 2.1 — Схема генетического алгоритма

Инициализация популяции

Формируется начальная популяция заданного размера. Каждая особь создаётся случайным образом или некоторым заданным алгоритмом.

Оценка приспособленности

Для каждой особи вычисляется значение фитнес-функции. Полученные значения позволяют ранжировать особей по качеству и используются на этапе селекции.

Селекция (отбор)

Из текущей популяции выбираются особи, которые станут родителями для создания потомства. Существуют различные алгоритмы селекции, отличающиеся случайностью и стойкостью выбора родителей. Алгоритм обычно устроен таким образом, чтобы лучшие особи имели больше шансов стать родителями, но при этом сохранялось определённое разнообразие для предотвращения преждевременной сходимости.

Кроссинговер (скрещивание)

Из отобранных родителей формируются пары, которые в результате применения оператора кроссинговера создают одну или две особи-потомка. В зависимости от реализации оператора может происходить обмен участками хромосом (аллелями), их математическая комбинация или иное преобразование, наследующее признаки родителей.

Мутация

Каждый ген полученных потомков с малой вероятностью подвергается случайному изменению. Мутация поддерживает генетическое разнообразие популяции и позволяет исследовать новые области пространства поиска, которые могли быть потеряны в процессе селекции и кроссинговера.

Проверка критерия остановки

Эволюционный процесс повторяется до выполнения одного из критериев:

- достижение максимального числа поколений;
- достижение требуемого значения фитнес-функции;
- отсутствие улучшений в течение заданного числа поколений (стагнация);
- превышение допустимого времени работы.

Таким образом, генетический алгоритм оперирует популяцией особей, каждая из которых кодирует потенциальное решение. Последовательное применение операторов селекции, кроссинговера и мутации позволяет эволюционировать популяции в направлении улучшения качества решений. В предлагаемом комбинированном методе генетический алгоритм планируется к использованию для адаптации параметров α и β муравьёв.

2.2 Описание предлагаемого метода

2.2.1 Требования к разрабатываемому методу

К разрабатываемому методу предъявляются следующие функциональные требования:

- возможность решения задачи коммивояжёра для полных взвешенных неориентированных графов;
- адаптация параметров муравьёв в процессе поиска.

и нефункциональные требования:

- улучшение сходимости по сравнению с классическим муравьиным алгоритмом;
- лучшая устойчивость к выбору начальных параметров α и β по сравнению с классическим муравьиным алгоритмом.

2.2.2 Формализация метода

На рисунке 2.2 представлено верхнеуровневое разбиение метода на последовательность действий. Блоки A1-A3 являются частями муравьиного алгоритма, а блок A4 описывает динамическую адаптацию параметров муравьёв.

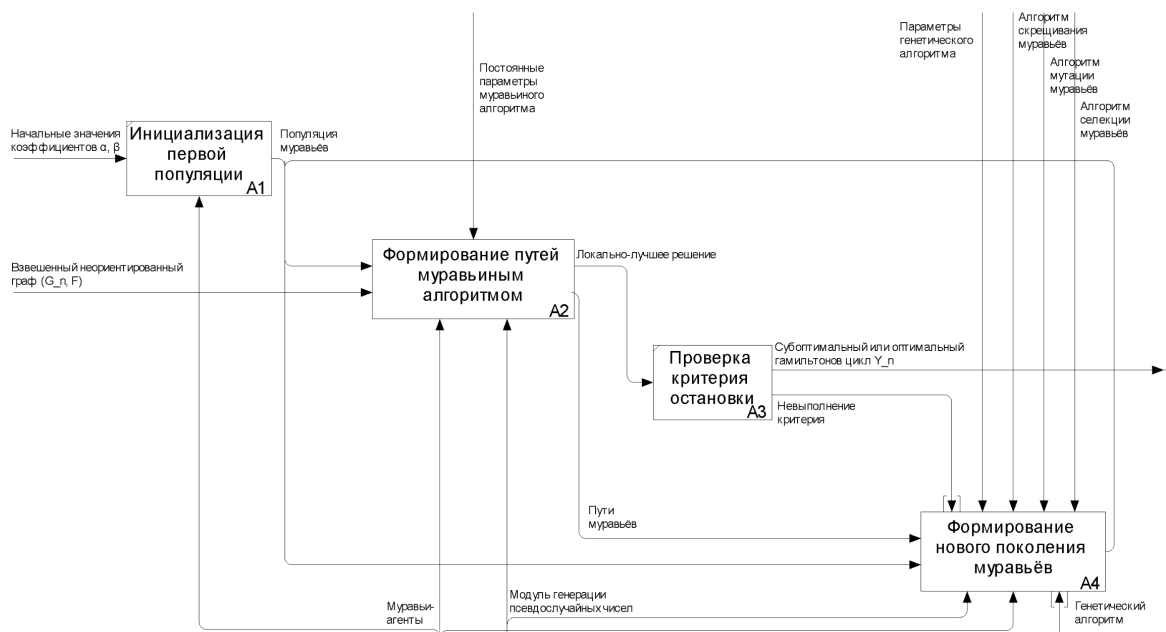


Рисунок 2.2 — IDEF0 диаграмма метода 1-го уровня

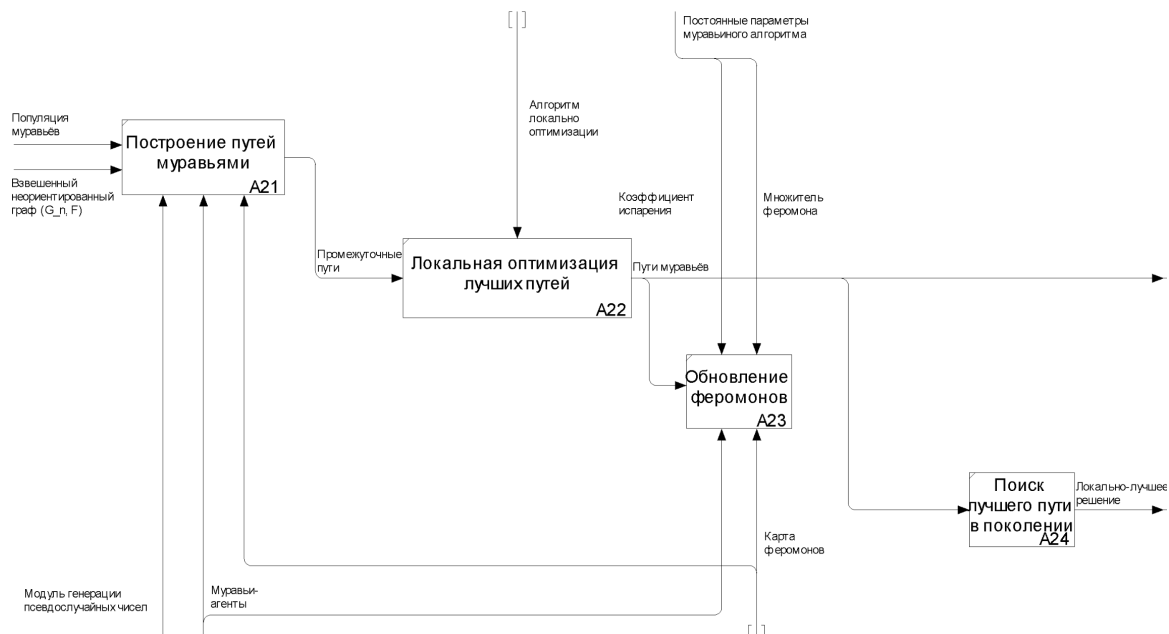


Рисунок 2.3 — IDEF0 диаграмма формирования путей муравьёв

Диаграмма декомпозиции блока A2 (рисунок 2.3) описывает процесс формирования путей муравьями, совпадающей с классическим муравьиным алгоритмом, с добавлением локальной оптимизации для избранных путей поколения.

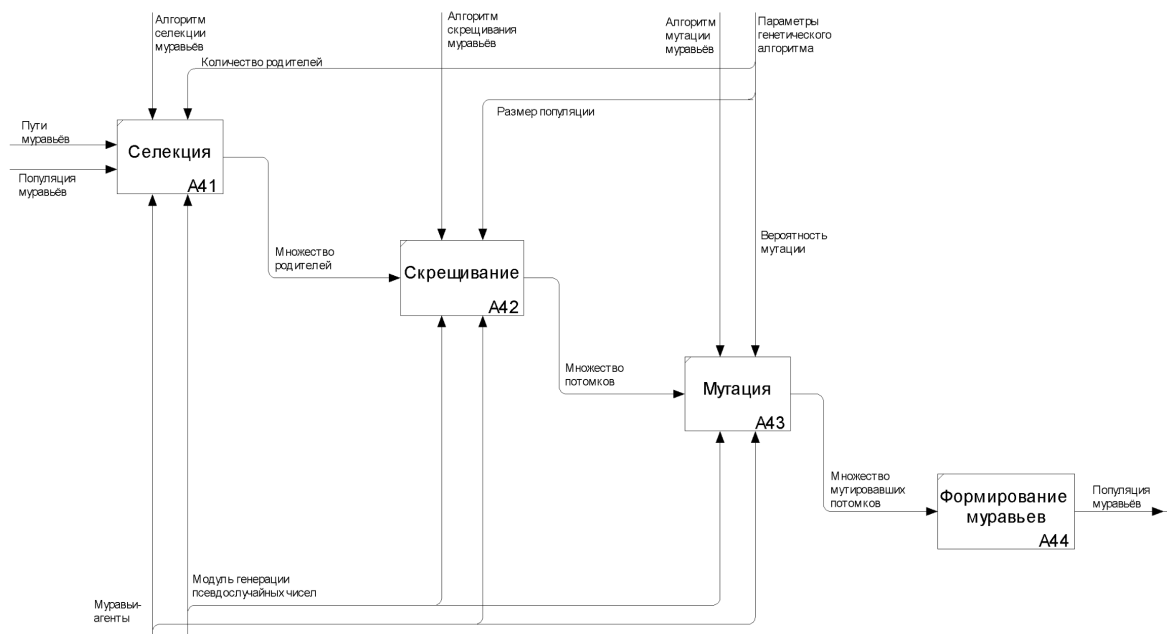


Рисунок 2.4 — IDEF0 диаграмма формирования новой популяции

Диаграмма декомпозиции блока A4 (рисунок 2.4) описывает работу генетического алгоритма со всеми его классическими этапами, однако особями в данном методе являются не решения задачи, а муравьи, в частности два их индивидуальных параметра: α и β .

Таким образом, параметрами метода, задаваемыми до начала работы являются:

- начальные значения α_0 и β_0 — коэффициенты первого поколения муравьёв;

- начальное количество феромона τ_0 – феромон, устанавливаемый для каждого ребра при инициализации карты;
- множитель феромона Q – определяет количество феромона распыляемую на единицу длины пути;
- коэффициент испарения p – определяет долю феромона, испаряющуюся в конце каждой итерации;
- количество итераций – параметр критерия остановки, равняющийся количеству поколений муравьёв
- размер популяции m – количество муравьёв, в одном поколении. Фактически, является количеством построенных решений в рамках одного поколения;
- период адаптации w – количество последовательных итераций, без обновления поколения;
- количество родителей – количество муравьёв, отбираемого для скрещивания
- параметры алгоритмов скрещивания, мутации, селекции.

Таким образом, генетический алгоритм интегрируется как метаэвристика над муравьиным алгоритмом, которая динамически адаптирует параметры муравьёв под конфигурацию графа и текущего уровня феромонов.

2.3 Поиск муравьиным алгоритмом

На основе представленной IDEF0-формализации и выделенных параметров метода ниже приводится детальное описание каждого этапа. Для каждого этапа описываются используемые алгоритмы, формулы и параметры, а также их обоснование с точки зрения цели разрабатываемого комбинированного подхода.

2.3.1 Инициализация первого поколения

Инициализация феромонов

Так же как и в классическом муравьином алгоритме на каждом ребре устанавливается в небольшое положительное значение уровня феромона:

$$\tau_{ij}(0) = \tau_0$$

Это позволяет сгладить первую итерацию, увеличивая шансы на выбор муравьём более длинных участков графа, что увеличивает начальную область поиска и разнообразие решений.

Формирование начальной популяции муравьёв

Создаётся популяция размером m , где m — количество муравьёв в одном поколении. Каждый муравей имеет индивидуальные α и β , равные некоторому установленному для всех значению по умолчанию.

2.3.2 Формирование путей муравьиным алгоритмом

Данный этап выполняется на каждой итерации для всех муравьёв, в результате чего находится множество и m решений.

Выбор начальной вершины

Для каждого муравья начальная вершина выбирается случайным образом. Это позволяет исследовать различные маршруты и снижает влияние начального выбора на итоговый результат.

Правило выбора очередной вершины

Вероятностное правило выбора следующей вершины пути аналогично классическому варианту с поправкой на гетерогенность параметров: Пусть на итерации k муравей a находится в вершине v_i , а множество ещё не посещённых им вершин обозначается как $U_a(i)$. Тогда вероятность выбора вершины $v_j \in U_a(i)$ в качестве следующей задаётся формулой:

$$P_{ij}^{(a)}(k) = \begin{cases} \frac{\tau_{ij}(k)^{\alpha_a} \cdot \eta_{ij}^{\beta_a}}{\sum_{v_l \in U_a(i)} \tau_{il}(k)^{\alpha_a} \cdot \eta_{il}^{\beta_a}}, & \text{если } v_j \in U_a(i); \\ 0, & \text{иначе,} \end{cases} \quad (2.1)$$

где

- $\tau_{ij}(k)$ – интенсивность феромона на ребре e_{ij} на итерации k ;
- $\eta_{ij} = \frac{1}{F(e_{ij})}$ – эвристическая значимость ребра (обратная длина);
- α_a и β_a – индивидуальные параметры муравья a , определяющие степень влияния феромона и эвристики соответственно.

Вычисление длин маршрутов

После того как все муравьи завершили построение своих маршрутов, происходит вычисление длины каждого цикла:

$$L_a(k) = F(Y_a^k), \quad (2.2)$$

где Y_a^k – цикл, построенный муравьём a .

Локальная оптимизация

Локальная оптимизация – метод, улучшающий уже найденное решение. Применение локальной оптимизации позволяет повысить качество решений без значительного увеличения вычислительных затрат [12]. В качестве кандидатов на применение локальной оптимизации предлагается брать родителей, выбираемых при селекции, так как эти особи уже отобраны как лучшие для скрещивания.

Наиболее популярной группой алгоритмов локальной оптимизации для задачи коммивояжёра являются k -орт алгоритмы, которые выполняют перестановку k рёбер в маршруте, замыкая его заново. При увеличении k качество улучшения растёт, однако k -орт имеет экспоненциальную вычислительную сложность $O(n^k)$, где n – число вершин графа, что делает k -орт нецелесообразным при $k > 3$ для муравьиного алгоритма.

В качестве метода локальной оптимизации в данной работе предлагается использовать 2-opt как наиболее сбалансированный по соотношению эффективности и вычислительных затрат [22].

Псевдокод алгоритма локальной оптимизации маршрута методом 2-opt [23] представлен в листинге 2.1.

Листинг 2.1 — Алгоритм 2-opt оптимизации маршрута

```

1 // маршрут — последовательность вершин в порядке обхода [v1, v2, ..., vn]
2 найдено_улучшение := ИСТИНА
3 Пока найдено_улучшение:
4     найдено_улучшение := ЛОЖЬ
5     Для i от 1 до n-1:
6         Для j от i+1 до n:
7             // проверка граничных условий
8             Если i+1 >= n или j+1 >= n:
9                 продолжить
10            Если i+1 = j или j+1 = i:
11                продолжить
12            // вычисляем изменение длины маршрута
13            текущ_расст := длина(маршрут[i], маршрут[j]) + длина(маршрут[i+1], маршрут[j+1])
14            новое_расст := длина(маршрут[i], маршрут[i+1]) + длина(маршрут[j], маршрут[j+1])
15            дельта = текущ_расст - новое_расст
16
17            Если дельта < 0:
18                // выполняем переворот участка между i+1 и j
19                переворот(маршрут, i+1, j)
20                найдено_улучшение := ИСТИНА

```

2.3.3 Обновление феромона

Обновление феромона в предлагаемом комбинированном методе проводится аналогично классическому муравьиному алгоритму в два этапа: испарение и добавление нового феромона.

Испарение феромона

На первом этапе происходит уменьшение концентрации феромона на всех рёбрах графа. Это моделирует естественный процесс рассеивания химического следа и позволяет алгоритму постепенно "забывать" устаревшую информацию, фокусируясь на более перспективных участках. Испарение выполняется по формуле:

$$\tau_{ij}(t+1) = (1 - \rho) \cdot \tau_{ij}(t),$$

где $\rho \in (0, 1]$ — коэффициент испарения, определяющий скорость уменьшения феромона; t — номер текущей итерации.

Добавление феромона

После испарения каждый муравей a добавляет феромон на рёбра, вошедшие в построенный им маршрут Y_a^k . Количество добавляемого феромона обратно пропорционально длине

маршрута, что усиливает более короткие пути:

$$\tau_{ij}(t+1) = \tau_{ij}(t) + \sum_{a=1}^m \Delta\tau_{ij}^{(a)},$$

$$\Delta\tau_{ij}^{(a)} = \begin{cases} \frac{Q}{L_a}, & \text{если ребро } (i, j) \text{ входит в маршрут муравья } a, \\ 0, & \text{иначе,} \end{cases}$$

где Q – множитель феромона; L_a – длина маршрута, построенного муравьём a .

2.3.4 Выбор значений параметров

Как уже было описано выше, муравьиный алгоритм существенно зависит от параметров, определяющих его поведение и эффективность. В книге [24] определили значения, дающие хороший результат на большом наборе входных данных:

- $\alpha - 1$;
- $\beta - 2-5$;
- множитель феромона $Q - 1$;
- коэффициент испарения $\rho - 0.3$;
- количество муравьёв $m - n$;
- начальный уровень феромона $\tau_0 - \frac{m}{C_{nn}}$;

где n – число узлов в графе, C_{nn} – длина маршрута, построенная некоторой элементарной эвристикой, например жадным алгоритмом. Указанные значения α и β могут служить как начальные в предлагаемом методе, а остальные использоваться как референсные значения.

2.4 Формирование новой популяции генетическим алгоритмом

Формирование новой популяции является ключевым этапом, отличающим предлагаемый комбинированный метод от классического муравьиного алгоритма. Именно на этом этапе происходит адаптация гетерогенных параметров α_a и β_a муравьёв под конкретную конфигурацию графа и текущую ситуацию поиска.

Новая популяция формируется из текущей с использованием генетического алгоритма и его классических: селекции, скрещивания и мутации. Процесс выполняется каждые w итераций (период адаптации), что позволяет муравьям построить маршруты с текущими параметрами для оценки их приспособленности и эффективности.

В качестве особей генетического алгоритма выступают муравьи, а в качестве хромосомы – вектор из двух вещественных чисел: параметров α_a и β_a . В качестве фитнес-функции муравья

предлагается величина, обратная длине его пути:

$$f_a = \frac{1}{L_a},$$

где L_a – длина маршрута, построенного муравьём a .

2.4.1 Алгоритмы селекции

Селекция предназначена для отбора родителей, которые будут участвовать в скрещивании. В разрабатываемом методе предлагаются 3 основные стратегии селекции [21].

Элитарная селекция

Данная стратегия является наиболее простой и прямолинейной – в качестве родителей отбирается заданное количество особей с наилучшими показателями приспособленности (наименьшей длиной маршрута). Такой способ гарантирует сохранение лучших генетических характеристик, однако он уменьшает пространство поиска, так как теряется разнообразие популяции.

Рулеточная селекция

Данный метод основан вероятностном подходе. Вероятность выбора особи пропорциональна её приспособленности. Чем лучше особь (чем короче построенный ею маршрут), тем больше вероятность быть выбранной в качестве родителя. Соответственно вероятность выбора муравья a как родителя равна:

$$P(a) = \frac{f_a}{\sum_{j=1}^m f_j},$$

где $f_a = \frac{1}{L_a}$ – приспособленность особи, а m – размер популяции.

Рулеточная селекция сохраняет разнообразие популяции, так как даже наихудшие особи имеют ненулевую вероятность быть выбранными, однако она чувствительна к масштабу значений фитнес-функции.

Турнирная селекция

Турнирный метод основан одновременно на случайности и выборе лучшего кандидата. Для каждого требуемого родителя случайным образом выбирается подмножество особей из популяции размером k (размер турнира), после чего из этого подмножества выбирается особь с наилучшим показателем приспособленности.

Турнирная селекция устойчива к масштабу значений фитнес-функции и позволяет регулировать давление отбора через параметр k , при этом наиболее частыми и удачными являются значения $k = 3$ и $k = 5$ [21]

2.4.2 Алгоритмы скрещивания

Скрещивание (кроссинговер) создаёт новых особей-потомков на основе генетической информации родителей. Так как хромосома в данном случае является вектором вещественных

чисел, то наиболее рациональным будет использование арифметических алгоритмов скрещивания. В разрабатываемом методе предлагается 3 часто используемых метода арифметического кроссинговера.

В рамках описания алгоритмов скрещивания, примем что уже выбраны некоторые родители a_1 и a_2 с параметрами $(\alpha_{a_1}, \beta_{a_1})$ и $(\alpha_{a_2}, \beta_{a_2})$ соответственно.

Арифметический кроссовер

Потомок a рождается как линейная комбинация родительских параметров:

$$\alpha_a = \lambda \cdot \alpha_{a_1} + (1 - \lambda) \cdot \alpha_{a_2}, \quad \beta_a = \lambda \cdot \beta_{a_1} + (1 - \lambda) \cdot \beta_{a_2},$$

Чаще всего берут $\lambda = 0.5$, что позволяет равномерно учитывать влияние обоих выбранных родителей

BLX-кроссовер (Blend Crossover)

Данный оператор создаёт потомка в интервале, расширенном относительно родительских значений.

Пусть $\alpha_{\min} = \min(\alpha_{a_1}, \alpha_{a_2})$, $\alpha_{\max} = \max(\alpha_{a_1}, \alpha_{a_2})$, а размах значений составляет $I_\alpha = \alpha_{\max} - \alpha_{\min}$. Тогда значение α_a для потомка равномерно выбирается из интервала:

$$\alpha_a \sim U[\alpha_{\min} - \gamma \cdot I_\alpha, \alpha_{\max} + \gamma \cdot I_\alpha],$$

где $U[a, b]$ – равномерная случайная величина на отрезке $[a, b]$, а γ – параметр BLX-кроссовера.

Аналогично для параметра β :

$$\beta_a \sim U[\beta_{\min} - \gamma \cdot I_\beta, \beta_{\max} + \gamma \cdot I_\beta],$$

При значении γ , равном 0, потомок выбирается строго между родительскими значениями. При положительном значении интервал расширяется за пределы родительских значений, что позволяет исследовать области, лежащие за пределами родительских характеристик. Согласно [21] при значении $\gamma = 0.5$ данный вид кроссинговера работает эффективнее арифметических кроссоверов.

SBX-кроссовер

В отличие от предыдущих методов, в которых особь единолично хранило генетическую информацию, в данном методе её хранит пара, особей, которые по отдельности могут существенно расходиться. Особенностью метода является то, что он имитирует точечный двоичный кроссовер для вещественных чисел и имеет следующие свойства:

- средние арифметические значения фитнес-функций потомков и родителей равны;
- сохраняется внешнее распределение решений.

Для заданных родителей создаются два потомка, с параметрами:

$$\alpha_1^a = 0,5 [(\alpha_{a_1} + \alpha_{a_2}) - \gamma \cdot (\alpha_{a_2} - \alpha_{a_1})], \quad \alpha_2^a = 0,5 [(\alpha_{a_2} + \alpha_{a_1}) + \gamma \cdot (\alpha_{a_2} - \alpha_{a_1})],$$

$$\beta_1^a = 0,5 [(\beta_{a_1} + \beta_{a_2}) - \gamma \cdot (\beta_{a_2} - \beta_{a_1})], \quad \beta_2^a = 0,5 [(\beta_{a_2} + \beta_{a_1}) + \gamma \cdot (\beta_{a_2} - \beta_{a_1})],$$

$$\gamma(u) = \begin{cases} (2u)^{\frac{1}{\eta+1}}, & \text{при } u(0,1) < 0.5, \\ (\frac{1}{2(1-u)})^{\frac{1}{\eta+1}}, & \text{при } u(0,1) \geq 0.5, \end{cases}$$

где $u(0,1)$ – равномерная случайная величина на отрезке $[0,1]$, а $\eta \in [2,5]$ – параметр разброса SBX, чем он больше, тем больше вероятность появления потомков с параметрами близкими к родительским. На некоторых конфигурациях генетического алгоритма данный алгоритм показывает большую эффективность относительно BLX [21], поэтому имеет смысл его рассмотреть.

2.4.3 Алгоритмы мутации

Мутация вносит случайные изменения в параметры потомков, поддерживая генетическое разнообразие популяции и позволяя исследовать новые области пространства поиска. Стандартным способом мутации хромосом вещественных чисел, является прибавление к ним некоторого небольшого случайного вещественного, что имитирует мутацию в естественной среде [21]

Равномерная мутация

Наиболее классическим и простым вариантом вещественной мутации является равномерная мутация, при котором к текущему значению гена прибавляется равномерно распределённая случайная величина в заданном диапазоне. Соответственно в данном методе мутация имеет вид:

$$\alpha' = \alpha + U[l, r], \quad \beta' = \beta + U[l, r],$$

где $U[a, b]$ – равномерная случайная величина на отрезке $[l, r]$, задаваемом как параметр алгоритма.

Гауссовская мутация

Арифметическая мутация является грубым оператором, так как она с равной вероятностью вносит изменения любой величины в пределах заданного интервала, что может приводить к чрезмерным отклонениям параметров и нарушению сходимости [21]. В рамках [19] предложили более естественный и плавный механизм мутации, показавшим себя более удачным выбором – гауссовская мутация. В этом случае мутация имеет вид:

$$\alpha' = \alpha + \mathcal{N}(\mu, \sigma), \quad \beta' = \beta + \mathcal{N}(\mu, \sigma),$$

где $\mathcal{N}(\mu, \sigma)$ – нормально распределённая случайная величина с математическим ожиданием μ и стандартным отклонением σ , задаваемым как параметр алгоритма.

2.5 Описание структур данных

Для реализации разрабатываемого комбинированного метода необходимо определить набор структур данных, обеспечивающих хранение и обработку информации о графе, феромонах, муравьях и популяции в целом. Ниже приведено описание каждой структуры и обоснование выбранных представлений.

2.5.1 Граф

Графы в решаемой задаче имеют следующие особенности:

- графы полные, т.е. для каждой пары вершин существует ребро;
- граф взвешенный – требуется хранить не только факт связи, но и вес;
- граф не изменяется в процессе решения задачи;
- частые операции итераций по смежным вершинам – при выборе следующей вершины муравьем;

Исходя из этого наиболее эффективной структурой данных для представления графа в данном методе является матрица смежности [25], так как она:

- обеспечивает доступ к весу ребра между любыми двумя вершинами за время $O(1)$, что критически важно для вычисления вероятностей перехода и эвристической привлекательности;
- матрица занимает n^2 ячеек памяти, независимо от количества рёбер, в отличие от других представлений, зависящих от количества рёбер;
- итерация по всем смежным вершинам возможно реализовать как итерацию по строке матрицы, без накладных расходов.

2.5.2 Карта феромонов

Карта феромонов служит для хранения концентрации феромона и фактически также является взвешенным графом. В отличие от входного графа, предполагает частые обновления весов (значений феромона) на отдельных ребрах.

Для представления карты феромонов также рационально выбрать матрицу смежности, так как она, помимо описанных выше особенностей, позволяет за $O(1)$ менять значение на отдельно взятом ребре, что критично для данной структуры.

2.5.3 Муравей

Муравей является агентом, строящим маршрут. Каждый муравей обладает следующими данными:

- α и β – вещественные числа, в разрабатываемом методе эти параметры гетерогенны поэтому хранятся индивидуально для каждого муравья;

- маршрут (путь) — последовательность вершин, отражающая порядок обхода городов;
- длина маршрута — сумма весов рёбер в маршруте;
- память (табу-список) — множество посещённых вершин, используемое для запрета повторного посещения;

Для представления муравья в памяти предполагается использование тип данных определяющих несколько полей в одну совокупность (структура, класс).

Выбор структуры данных для представления маршрута должен исходить из следующих его особенностей:

- маршрут имеет фиксированную длину, равную количеству вершин графа n ;
- частое добавление вершин в конец маршрута в процессе построения;
- важно сохранение порядка элементов;
- редкое полное копирование маршрута (при сохранении лучшего решения);
- необходимость вычисления длины маршрута путём суммирования весов рёбер.

Исходя из этого, для представления маршрута предлагается использовать массив фиксированного размера n . Такой выбор обеспечивает:

- сохранение порядка за счёт индексации массива;
- простоту копирования при необходимости сохранить лучший маршрут;
- компактное хранение (без накладных расходов на динамические структуры);
- возможность предварительного выделения памяти.

Табу-список используется на этапе выбора следующей вершины пути муравья и обладает следующими особенностями:

- частые операции проверки наличия элемента (n^2 операций для одного муравья);
- количество элементов фиксировано и равно n ;
- операции добавления вершины в список выполняются после каждого перехода.

Для реализации табу-списка предлагается использовать **хеш-таблицу (множество)** [26].

Такой подход обеспечивает:

- проверку наличие вершины в списке за $O(1)$;
- добавление элемента за $O(1)$;
- возможность предварительного выделения структуры под заданное число элементов.

2.5.4 Популяция муравьёв

Популяция представляет собой совокупность муравьёв, действующих в рамках одного поколения. Основные особенности популяции:

- количество муравьёв фиксированно;
- не важен порядок муравьёв;
- важно возможность быстрой очистки структуры, перед новым заполнением;
- важна возможность быстрой сортировки по длине построенных маршрутов (для неко-

торых вариантов селекции).

Исходя из этого, для представления популяции предлагается использовать **массив (срез)** муравьёв фиксированного размера m . Такой выбор обеспечивает:

- прямой доступ к любому муравью по индексу за $O(1)$;
- возможность быстрой сортировки с использованием стандартных алгоритмов (например, быстрой сортировки хоара [26] за $O(m \log(m))$);
- простоту очистки — достаточно перезаписать массив новыми данными или создать новый;
- компактное хранение без накладных расходов на указатели (в отличие от связанных списков).

2.5.5 Итоговый выбор структур данных

Результаты выбора структур данных для реализации разрабатываемого метода сведены в таблицу 2.1.

Таблица 2.1 — Выбор структур данных для реализации метода

Структура	Выбранное представление	Обоснование
Граф	Матрица смежности	Доступ к весу ребра за $O(1)$ полный граф
Карта феромонов	Матрица феромонов (смежности)	Доступ и обновление за $O(1)$, полный граф
Маршрут муравья	Массив фиксированного размера	Фиксированная длина, добавление в конец за $O(1)$, простота копирования
Табу-список	Хеш-таблица (множество)	Проверка и добавление за $O(1)$
Популяция	Массив фиксированного размера	Прямой доступ, быстрая сортировка, простота очистки

2.6 Архитектурное разбиение на модули

Для разграничения зон ответственности и возможности гибкой конфигурации метода, предлагается архитектура с разбиением на отдельные модули и компоненты, описанная в UML диаграмме 2.5

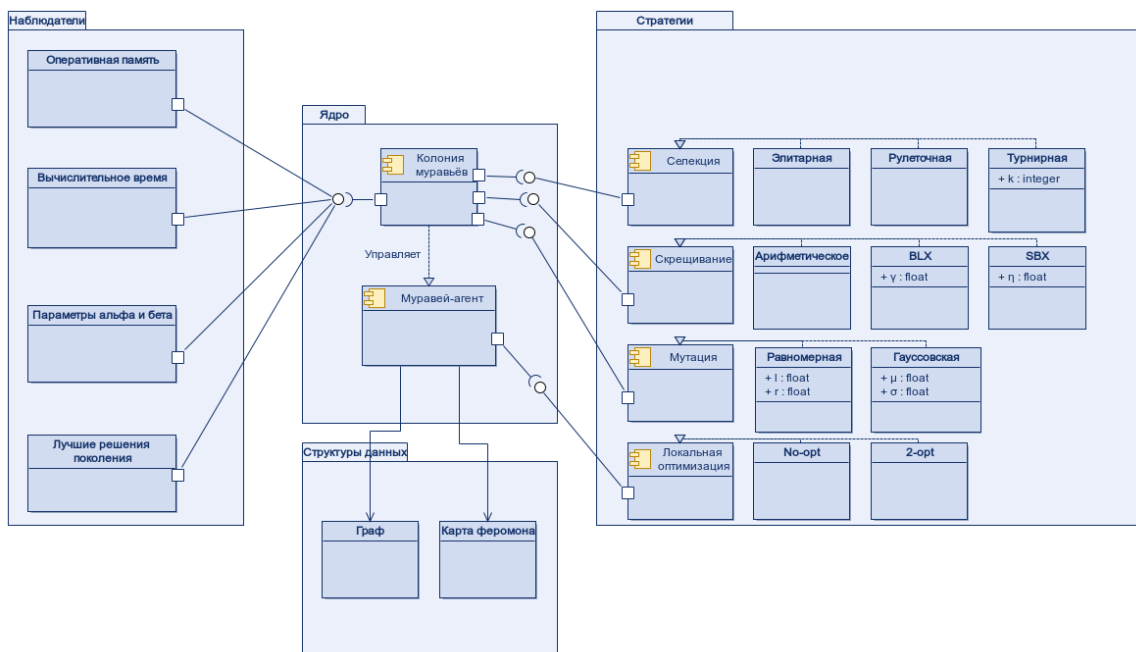


Рисунок 2.5 — UML-диаграмма модулей и компонентов предлагаемого метода

2.6.1 Модуль структуры данных

Модуль структур данных отвечает за описание абстрактных типов данных и их реализацию, для использования в ядре метода. Данный модуль не содержит логики структур, а только предоставляет интерфейсы для доступа к данным.

Состоит из двух сущностей, реализующих соответствующие структуры данных:

- граф;
- карта феромонов.

2.6.2 Модуль ядро

Модуль ядра реализует основную логику комбинированного метода и является центральным компонентом архитектуры. Он не зависит от конкретных реализаций стратегий, а определяет интерфейсы, что позволяет гибко конфигурировать поведение метода.

Состоит из двух основных компонентов:

Колония муравьев

- содержит популяцию муравьёв, граф и карту феромонов;
- управляет итерационным процессом: построение маршрутов, обновление феромонов, эволюция популяции, сохранением лучшего пути;
- передает срезы информации в определенные моменты наблюдателям, для сбора информации о процессе выполнения;
- реализует критерии остановки алгоритма;
- определяет интерфейсы для стратегий генетического алгоритма и наблюдателей.

Муравей-агент

- хранит индивидуальные параметры α и β ;
- определяет алгоритм построения маршрута;
- определяет интерфейс метода локальной оптимизации и применяет его;
- вычисляет длину построенного маршрута.

Таким образом колония муравьев является управляющим элементом, задающим общие шаги метода, при этом построение пути и распыление феромона ложится на самих муравьев, а конкретные вариации генетических операторов легко подменяются за счёт определенных интерфейсов на этапе инициализации.

2.6.3 Модуль стратегии

Модуль стратегий содержит конкретные реализации интерфейсов, определённых в ядре. Вынесение стратегий в отдельный модуль реализует паттерн проектирования «Стратегия» [27], который позволяет:

- отделить алгоритмы от использующего их контекста (ядро метода);
- подменять алгоритмы на этапе конфигурации без изменения кода ядра;
- добавлять новые алгоритмы без модификации существующих компонентов.

Таким образом, все вариативные части метода (алгоритмы селекции, скрещивания, мутации, выбора пути, обновления феромонов, локальной оптимизации) вынесены в модуль стратегий. Ядро же содержит только неизменяемую логику работы колонии муравьёв — последовательность этапов и управление итерационным процессом.

2.6.4 Модуль наблюдатели

Модуль наблюдателей реализует шаблон проектирования «Наблюдатель» [27], позволяя отслеживать состояние алгоритма в реальном времени без вмешательства в его логику. Все наблюдатели реализуют интерфейс, определённый в ядре, что позволяет легко модифицировать собираемую информацию.

Механизм работы модуля наблюдателей построен на следующих принципах:

- ядро определяет интерфейс наблюдателя и передаваемую им информацию о текущем состоянии метода;
- при начальной конфигурации колония регистрирует список наблюдателей;
- после каждой итерации колония собирает текущее состояние и уведомляет всех зарегистрированных наблюдателей, передавая им срез текущего состояния (популяция муравьёв, карта феромонов, лучшее решение, номер итерации и т.д.).

Помимо передаваемой информации, наблюдатели могут собирать некоторую информацию в момент вызова о среде выполнения и операционной системе, накапливать некоторые значения и т.д.

Для реализации предлагаются следующие варианты наблюдателей:

- **наблюдатель лучшего решения** — на каждой итерации сохраняет лучшее решение

текущего поколения и его длину. Позволяет провести анализ того, как менялся результат работы метода;

— **наблюдатель средних параметров** — вычисляет и сохраняет средние значения параметров α и β в популяции на каждой итерации; позволяет анализировать эволюционные процессы в колонии;

— **наблюдатель использования памяти** — собирает информацию об объёме выделенной оперативной памяти в процессе работы;

— **наблюдатель времени выполнения** — измеряет длительность каждой итерации и общее время работы метода.

Вывод

В результате конструкторской части были сформулированы требования к разрабатываемому методу, представлена его формализация в нотации IDEF0 и детально описаны все этапы работы: инициализация, построение маршрутов, обновление феромонов, формирование новой популяции с использованием операторов селекции, скрещивания и мутации. Выбраны и обоснованы структуры данных для реализации метода (матрица весов для графа, матрица феромонов, массив для популяции и маршрута, хеш-таблица для табу-списка). Предложена модульная архитектура, разделяющая компоненты на модули структур данных, ядра, стратегий и наблюдателей, что обеспечивает гибкость конфигурации и расширяемость метода. Разработанный метод сочетает муравьиный алгоритм с эволюцией гетерогенных параметров α и β на основе генетического алгоритма, что позволяет адаптировать поведение муравьёв под конкретную конфигурацию графа.

3 Технологическая часть

3.1 Обоснования выбора программных средств реализации

3.1.1 Формат входных и выходных данные

Входными данными метода является взвешенный граф, описывающий конфигурацию задачи коммивояжёра. В качестве вводного формата для графа был выбран формат TSPLIB [28], так как:

- TSPLIB является стандартом для тестирования алгоритмов решения задачи коммивояжёра и имеет множество готовых конфигураций, по которым в том числе можно сравнить метод с другими;
- формат поддерживает различные типы весов рёбер и графов;
- предоставляет обширную библиотеку готовых тестовых задач различной размерности;

Выходными данными являются:

- **гамильтонов цикл** — последовательность вершин в порядке обхода, начинающаяся и заканчивающаяся в одной и той же вершине. Форматом вывода для цикла может служить строковая последовательность меток узлов графа или графическая визуализация;
- **длина найденного маршрута** — сумма весов всех рёбер, входящих в найденный цикл. В зависимости от типа графа, может быть либо целым, либо вещественным числом.

3.1.2 Требования предъявляемые к ПО

Разрабатываемое программное обеспечение должно реализовывать предложенный комбинированный метод и предоставлять графический интерфейс для взаимодействия с ним. Исходя из форматов входных и выходных данных к ПО предъявляются следующие требования:

- возможность загрузки графа из файла формата TSPLIB, описывающего полный взвешенный неориентированный граф;
- возможность задания параметров комбинированного метода;
- возможность задания различных стратегий и их индивидуальных параметров;
- визуализация найденного гамильтонова цикла в графическом виде для случаев двумерной евклидовой плоскости;
- визуализация характеристик метода (затрачиваемая оперативная память, время вычисления) и истории изменения длин путей и параметров α , β в процессе поиска решения.

3.1.3 Язык программирования

В качестве основного языка для реализации разрабатываемого метода был выбран язык Go [29], так как:

- средства языка достаточно для реализации всех спроектированных алгоритмов и структур данных;
- в стандартной библиотеке присутствуют необходимые структуры данных (массивы, хеш-таблицы) и алгоритмы (сортировка, генерация случайных чисел);
- язык позволяет разделение программы на модули и описание интерфейсов между ними.

3.1.4 Интерфейс

Для взаимодействия с разработанным методом принято решение реализовать веб-интерфейс, так как:

- средств браузера достаточно, для ввода и визуализации данных, решения задачи;
- веб-интерфейс не требует установки дополнительного программного обеспечения на стороне пользователя.

Как следствие такого выбора, интерфейс делится на две части: серверную и клиентскую. Для реализации серверной части был выбран фреймворк Gin [30]. В качестве формата клиентского формата был выбран SSR, а для клиентской — подход Server-Side Rendering (SSR), при котором HTML-страницы генерируются на сервере из шаблонов, для которых выбрана библиотека templ. Клиентская логика (обработка ввода пользователя, отправка запросов к API, визуализация результатов) реализуется на языке TypeScript [31] с использованием библиотеки plotly.js [32] для построения графиков найденных маршрутов и динамических характеристик.

3.2 Особенности реализации

3.2.1 Муравей

В листинге 3.1 представлена структура муравья, хранящая индивидуальные параметры α и β , что обеспечивает гетерогенность колонии. Табу-список реализованный с помощью встроенной хеш-таблицы go, а маршрут – срезом.

Листинг 3.1 — Структура муравья

```

1 type HeteroAnt struct {
2     // Configuration parameters
3     alpha          float64
4     beta           float64
5     ...
6     // Runtime state (initialized in Prepare)
7     ...
8     current *graph.Vertex
9     path     []*graph.Vertex
10    visited  map[*graph.Vertex] struct{}
11    ...
12 }
```

Алгоритм построение маршрута представлен в листинге 3.2 : на каждом шаге муравей

выбирает следующую вершину согласно вероятностному правилу, учитывающему его индивидуальные α и β .

Листинг 3.2 — Цикл поиска пути муравьём

```
1 func (a *HeteroAnt) Run() error {
2     ...
3     for !a.step() {
4         // Continue until step returns true (no more vertices)
5     }
6     a.done = true
7     return nil
8 }
9
10 func (a *HeteroAnt) step() bool {
11     next := a.chooseNext.ChooseNext(a)
12     if next == nil {
13         return true // Path complete
14     }
15     ... // обновление пути и текущей вершины
16     return false
17 }
18
19 func (s *PathClassicStrategy) ChooseNext(ant ant.AntView) *graph.Vertex {
20     ...
21     g.ForEachSource(current, func(edge *graph.Edge) bool {
22         t := edge.Target()
23         if ant.Visited(t) {
24             return false
25         }
26         w := math.Pow(1./edge.Weight(), ant.Beta())
27         p := math.Pow(pm.Get(edge), ant.Alpha())
28         probability := w * p
29         accumulatedProbability += probability
30         candidates = append(candidates, NextCandidate{
31             v:      t,
32             probability: probability,
33         })
34         return false
35     })
36     ...
37     r := rand.Float64() * accumulatedProbability
38
39     cumulative := 0.
40     for _, candidate := range candidates {
41         cumulative += candidate.probability
42         if r <= cumulative {
43             return candidate.v
44         }
45     }
46     ...
47 }
```

После завершения построения всех маршрутов, муравьи распыляют феромон на пройденные рёбра согласно алгоритму, представленному на листинге 3.3. Вызов метода происходит из управляющей структуры – колонии муравьёв.

Листинг 3.3 — Распыление феромона

```

1 func (*ApplyClassicStrategy) ApplyPheromone(ant ant.AntView) {
2     ...
3     delta := ant.PheromoneMultiplier() / ant.Score()
4     for i := 0; i < len(path)-1; i++ {
5         s, t := path[i], path[i+1]
6         e, _ := g.Edge(s, t)
7         pm.Add(e, delta)
8     }
9     wrapE, _ := g.Edge(path[len(path)-1], path[0])
10    pm.Add(wrapE, delta)
11 }

```

3.2.2 Локальная оптимизация

Для повышения качества найденных решений и ускорения сходимости в методе может применяться локальная оптимизация маршрутов. На листинге 3.4 представлена реализация алгоритма 2-орт, которая применяется к отобранным родителям перед формированием нового поколения.

Листинг 3.4 — 2-opt оптимизация

```

1 func (s TwoOptLocalOptimisation) Optimise(path []graph.Vertex, g *graph.Graph) {
2     n := len(path)
3     improved := true
4     for improved {
5         improved = false
6         for i := 0; i < n-2; i++ {
7             for j := i + 2; j < n-1; j++ {
8                 e1, _ := g.Edge(path[i], path[i+1])
9                 e2, _ := g.Edge(path[j], path[j+1])
10
11                 if s.shouldSwap(e1, e2, g) {
12                     s.swapEdges(path, i, j)
13                     improved = true
14                     break
15                 }
16             }
17             if improved {
18                 break
19             }
20         }
21     }
22 }
23
24 func (s *TwoOptLocalOptimisation) shouldSwap(e1, e2 *graph.Edge, g *graph.Graph) bool {
25     v1, v2 := e1.Source(), e1.Target()
26     u1, u2 := e2.Source(), e2.Target()
27     // Текущая длина двух рёбер
28     curWeight := e1.Weight() + e2.Weight()
29     // Новая длина после перестановки
30     newWeight := g.Edge(v1, u1).Weight() + g.Edge(v2, u2).Weight()
31     return newWeight < curWeight // замена сокращает маршрут
32 }
33
34 func (s TwoOptLocalOptimisation) swapEdges(path []graph.Vertex, i, j int) {
35     // Переворот участка между i+1 и j
36     for start, end := i+1, j; start < end; start, end = start+1, end-1 {

```

```

37     path[start], path[end] = path[end], path[start]
38 }
39 }

```

3.2.3 Колония

Колония (листинг 3.5) является центральным управляющим компонентом метода. Она хранит популяцию муравьёв, карту феромонов, а также настройки и стратегии генетического алгоритма. Поля `parentSelect`, `crossover`, `mutation` представляют собой стратегии селекции, скрещивания и мутации соответственно. Срез `observers` содержит зарегистрированных наблюдателей для сбора статистики.

Листинг 3.5 — Структура колонии

```

1
2 type HeteroAntColony struct {
3     // Стратегии
4     parentSelect      ParentSelectionStrategy
5     crossover          CrossoverStrategy
6     mutation          MutationStrategy
7     // Параметры муравьиного алгоритма
8     defaultAlpha       float64
9     defaultBeta        float64
10    pheromoneMultiplier float64
11    evaporationRate     float64
12    initialPheromone    float64
13    // Параметры генетического алгоритма
14    generationCount     uint
15    colonySize          uint
16    generationPeriod    uint
17    parentCount         uint
18    ...
19    best *ant.HeteroAnt
20    score float64
21    ...
22    observers [] ColonyObserver // Наблюдатели
23 }

```

Основной цикл метода представлен в листинге 3.6, реализует итерационный процесс:

- на каждой итерации все муравьи строят маршруты;
- каждые w итераций (период адаптации `generationPeriod`) происходит отбор родителей заданной стратегией, к которым применяется локальная оптимизация 2-орт;
- после построения маршрутов выполняется испарение феромонов;
- затем муравьи распыляют феромон на пройденные рёбра;
- наблюдатели уведомляются о текущем состоянии для сбора статистики;
- если на данной итерации выполнялась селекция, формируется новое поколение, где последовательно применяются скрещивание и мутация, переданные при конфигурации.

Листинг 3.6 — Основной цикл метода

```

1 func (c *HeteroAntColony) Run() error {
2     for i := uint(0); i < c.colonySize; i++ {...} // Инициализация первого поколения
3     // Основной цикл

```

```

4   for gen := uint(0); gen < c.generationCount; gen++ {
5       for _, a := range c.ants {...a.Run()...} // Построение путей муравьями
6       // Отбор
7       var parents []ant.AntView
8       if gen%c.generationPeriod == 0 {
9           ...
10          parents = c.parentSelect.SelectParents(views, c.parentCount)
11          for _, p := range parents {...a.LocalOptimisation(c.localOptimisation)...} // Локальная
              оптимизация
12          ...
13      }
14      // Поиск более лучшего пути среди найденных
15      for _, a := range c.ants {
16          if c.best == nil || bestInGen.Score() < c.best.Score() {
17              c.best = a
18          }
19      }
20      c.evaporatePheromones() // Испарение
21      for _, a := range c.ants {...a.ApplyPheromone()...} // Нанесение феромонов
22      // Уведомление наблюдателей
23      dto := &ColonyObserverDTO {...}
24      for _, obs := range c.observers {
25          obs.Observe(dto)
26      }
27
28      // Обновление поколения генетическим алгоритмом
29      if parents != nil {
30          c.prepareNextGeneration(parents)
31      }
32  }
33  return nil
34  }
35  // Функция испарения феромонов
36  func (c *HeteroAntColony) evaporatePheromones() {
37      factor := 1 - c.evaporationRate
38      c.g.ForEachEdge(func(e *graph.Edge) bool {
39          c.pm.Update(e, c.pm.Get(e)*factor)
40          return false
41      })
42  }
43  func (c *HeteroAntColony) prepareNextGeneration(parents []ant.AntView) {
44      // Скрещивание
45      offspring := make([]*ant.HeteroAnt, 0, c.colonySize)
46      for len(offspring) < int(c.colonySize) {
47          p1, p2 := parents[rand.Intn(len(parents) - 1)], parents[rand.Intn(len(parents) - 1)]
48          offspring = append(offspring, c.crossover.Crossover(p1, p2)...)
49      }
50      // Мутация
51      for i := 0; uint(i) < c.colonySize; i++ {
52          offspring[i] = c.mutation.Mutate(offspring[i])
53      }
54
55      // Замена
56      c.ants = offspring
57  }

```


3.2.4 Стратегии

В листинге 3.7 представлены интерфейсы стратегий генетического алгоритма, которые используются колонией для эволюции популяции. Как видно, параметры мутации хранятся напрямую в её структуре и передаются при создании её экземпляра, что даёт возможность не включать всевозможные параметры стратегий в основную структуру колонии.

Листинг 3.7 — Интерфейсы стратегий

```
1 type ParentSelectionStrategy interface {
2     SelectParents(ants [] ant.AntView, n uint) [] ant.AntView
3 }
4
5 type CrossoverStrategy interface {
6     Crossover(p1, p2 ant.AntView) []* ant.HeteroAnt
7 }
8
9 type MutationStrategy interface {
10     Mutate(ant ant.AntView) *ant.HeteroAnt
11 }
```

Как пример реализации одного из интерфейсов, в листинге 3.8 приведена реализация одного из видов мутации – гауссовской.

Листинг 3.8 — Гауссова мутация

```
1 type GaussMutationStrategy struct {
2     sigma float64
3     mu     float64
4 }
5
6 var _ colony.MutationStrategy = (*GaussMutationStrategy)(nil)
7
8 func normalRand(sigma, mu float64) float64 {
9     return rand.NormFloat64()*sigma + mu
10 }
11
12 func NewGaussMutationStrategy(sigma, mu float64) *GaussMutationStrategy {
13     return &GaussMutationStrategy{
14         sigma: sigma,
15         mu:     mu,
16     }
17 }
18
19 func (s *GaussMutationStrategy) Mutate(a ant.AntView) *ant.HeteroAnt {
20     alpha := normalRand(s.sigma, s.mu)
21     beta := normalRand(s.sigma, s.mu)
22
23     return ant.NewHeteroAnt(
24         a.Alpha()+alpha,
25         a.Beta()+beta,
26         a.PheromoneMultiplier(),
27         a.PathStrategy(),
28         a.PheromoneApplyStrategy(),
29     )
30 }
```

3.2.5 Наблюдатели

Для сбора статистики и мониторинга работы алгоритма в методе реализован паттерн «Наблюдатель». Интерфейс `ColonyObserver` (листинг 3.9) содержит метод `Observe`, принимающий структуру-передачу данных `ColonyObserverDTO`. Она инкапсулирует текущее состояние колонии: популяцию муравьёв, карту феромонов, лучшее найденное решение и номер текущей итерации и позволяет наблюдателем агрегировать эти данные для статистики.

Листинг 3.9 — Интерфейс наблюдателя

```
1 type ColonyObserverDTO struct {
2     C      *HeteroAntColony
3     Ants   []ant.AntView
4     Pm     *pheromone.PheromoneMap
5     G      *graph.Graph
6     Best   *ant.HeteroAnt
7     Score  float64
8
9     Generation uint
10 }
11
12 type ColonyObserver interface {
13     Observe(dto *ColonyObserverDTO)
14 }
```

В листинге 3.10 представлен пример наблюдателя – `TimeObserver`, предназначенный для замера времени выполнения. Он сохраняет временные метки для каждой итерации, что позволяет впоследствии проанализировать производительность алгоритма.

Листинг 3.10 — Наблюдатель времени выполнения

```
1 type RunTime struct {
2     Run      uint
3     Moment   time.Time
4     Time     time.Duration
5 }
6
7 type TimeObserver struct {
8     startTime time.Time
9     endTime   time.Time
10    runs       []RunTime
11 }
12
13 func NewTimeObserver(gens uint) *TimeObserver {
14     return &TimeObserver{
15         runs: make([]RunTime, 0, gens),
16     }
17 }
18
19 var _ colony.ColonyObserver = (*TimeObserver)(nil)
20
21 func (o *TimeObserver) Observe(dto *colony.ColonyObserverDTO) {
22     if len(o.runs) == 0 {
23         o.runs = append(o.runs, RunTime{
24             Run:      dto.Generation,
25             Moment:   time.Now(),
26             Time:     time.Since(o.startTime),
```

```
27     })
28     return
29 }
30 lastRun := o.runs[len(o.runs)-1]
31 o.runs = append(o.runs, RunTime{
32     Run:    dto.Generation,
33     Moment: time.Now(),
34     Time:    time.Since(lastRun.Moment),
35 })
36 }
```

3.3 Тестирование

3.4 Пример работы ПО

Вывод

4 Исследовательская часть

Объектом данного исследования является качество решений полученных разработанным методом.

4.1 Исследование вариантов метода

4.1.1 Цель исследования

Объектом данного исследования является разработанный муравьиный алгоритм с динамическими гетерогенными параметрами на основе генетического алгоритма.

Предметом данного исследования является зависимость качества получаемых алгоритмом решений в зависимости от конфигурации эволюционных операторов.

Цель исследования — определить оптимальную конфигурацию эволюционных операторов, обеспечивающую наилучшее качество решений задач коммивояжёра разработанным алгоритмом.

4.1.2 Описание исследования

Варьируемыми факторами исследования являются:

- стратегия селекции – элитарная, турнирная, рулеточная;
- стратегия скрещивания – арифметическая, BLX, SBX;
- стратегия мутации – равномерная, гауссовская.

Таким образом требуется сравнить $3 \times 3 \times 2 = 18$ конфигураций разработанного алгоритма.

Также варьируется входной взвешенный неорграф, описывающий задачу коммивояжёра. Были выбраны 6 тестовых задач из библиотеки TSPLIB: ulysses22.tsp, swiss42.tsp, eil51.tsp, berlin52.tsp, st70.tsp, eil76.tsp. Данные задачи различаются размерностью (от 22 до 76 городов), топологией и методом задания весов, что позволяет оценить алгоритм на различных типах входных данных.

Постоянными параметрами и их значениями в данном исследовании являются:

- начальные значения α и β : 1 и 3 соответственно;
- коэффициент испарения p : 0.5;
- множитель феромона Q : 1;
- количество муравьёв m : n
- начальный уровень феромона τ_0 : $\frac{m}{C_{nn}}$
- количество поколений: 300;
- период адаптации w : 5;
- количество родителей: $0.4m$;

где n – количество вершин в графе, C_{nn} – длина маршрута, построенная жадным алгоритмом. Для параметров эволюционных стратегий были выбраны следующие значения:

- равномерная мутация: от -1 до 1;
- гауссова мутация: $\sigma = 0.05$, $\mu = 0$;
- турнирная селекция: $k = 5$;
- BLX-скрещивание: $\gamma = 0.5$;
- SBX-скрещивание: $\eta = 2$.

Для сравнения конфигураций предлагаются следующие метрики:

Статистические оценки ошибки

Относительная ошибка ε – отношение разности длины найденного маршрута с лучшим известным решением для данного графа к этому же лучшему решению:

$$\varepsilon = \frac{|L - L|}{L}$$

Средняя относительная ошибка μ_{err} – среднее арифметическое относительной ошибки по всем графам и запускам:

$$\mu_{err} = \frac{1}{R * (|G| - 1)} \sum_{r=1}^R \sum_g^G \varepsilon_{rg},$$

где R – количество запусков для заданной конфигурации и графа, G – множество входных графов;

Среднеквадратичное отклонение относительной ошибки σ_{err} — показатель стабильности алгоритма:

$$\sigma_{err} = \sqrt{\frac{1}{R * (|G| - 1)} \sum_{r=1}^R \sum_g^G (\varepsilon_{rg} - \mu_{err})^2}$$

Использование относительной ошибки позволяет нивелировать различные размерности графов и оценивать их единым образом. μ_{err} и σ_{err} фактически являются оценками мат. ожидания и стандартного отклонения относительной ошибки метода в заданной конфигурации.

Ранги конфигураций

Как метрику для сравнения конфигураций между собой предлагается использовать средний ранг. Для каждого графа g все конфигурации упорядочиваются по возрастанию длины найденного ими решения. Лучшей конфигурации на данном графе присваивается ранг $r_g = 1$, следующей — ранг 2 и так далее до $N = 18$

Средний ранг конфигурации вычисляется как среднее арифметическое её рангов по всем графам:

$$\bar{r} = \frac{1}{|G|} \sum_{g=1}^G r_g$$

Ранжирования позволит оценить качество решений полученных конфигураций устойчиво относительно масштаба и выбросов: на каждой задаче конфигурации сравниваются только между собой, что важно при объединении результатов с различных графов, имеющих разную

размерность и структуру.

Алгоритм исследования

- 1) Для каждой тестовой задачи:
 -) загрузить граф из TSP-файла;
 -) вычислить жадный тур для инициализации феромонов;
- 2) Для каждой из 18 конфигураций:
 -) выполнить R независимых запусков алгоритма;
 -) для каждого запуска зафиксировать лучшее найденное решение (score);
- 3) Для каждой задачи и каждой конфигурации:
 -) вычислить относительную ошибку ε_{rg} ;
 -) рассчитать среднее μ_{err} и стандартное отклонение σ_{err} по R запускам;
 -) рассчитать ранг конфигурации по задаче;
- 4) Для каждой конфигурации:
 -) рассчитать средний ранг по всем задачам (ранг 1 присваивается лучшему методу на задаче);

4.1.3 Результаты исследования

Исследование проводилось на устройстве со следующими характеристиками:

- Процессор: AMD Ryzen 7 5800H (16 ядер, 4.46 ГГц);
- Оперативная память: 16 ГБ;
- Операционная система: openSUSE Tumbleweed x86_64 (Linux 6.19.12-1-default).

Для каждой конфигурации эволюционных операторов и входного графа алгоритм запускался $R = 20$ раз. Усреднённые результаты представлены в таблица 4.3-4.8. Результаты ранжирования и расчёта относительных ошибок представлены в таблице 4.1.

Таблица 4.1 — Ранжирование алгоритмов по среднему рангу

№	Селекция	Кроссинговер	Мутация	Средний ранг	μ_{err}	σ_{err}
1	Рулеточная	BLX	Гауссовская	4.5	0.042	0.010
2	Турнирная	Арифметический	Гауссовская	4.7	0.043	0.012
3	Турнирная	Арифметический	Равномерная	5.5	0.043	0.012
4	Элитарная	Арифметический	Равномерная	6.8	0.044	0.010
5	Элитарная	Арифметический	Гауссовская	7.0	0.044	0.012
6	Рулеточная	BLX	Равномерная	8.3	0.045	0.014
7	Рулеточная	SBX	Равномерная	8.8	0.045	0.013
8	Рулеточная	Арифметический	Гауссовская	9.0	0.046	0.013
9	Рулеточная	SBX	Гауссовская	9.5	0.045	0.012
10	Турнирная	SBX	Равномерная	10.7	0.047	0.013

Таблица 4.1 — Ранжирование алгоритмов (продолжение)

№	Селекция	Кроссинговер	Мутация	Средний ранг	μ_{err}	σ_{err}
11	Турнирная	BLX	Гауссовская	10.7	0.047	0.013
12	Турнирная	SBX	Гауссовская	10.8	0.046	0.013
13	Рулеточная	Арифметический	Равномерная	10.8	0.047	0.015
14	Элитарная	SBX	Гауссовская	11.2	0.046	0.013
15	Турнирная	BLX	Равномерная	11.7	0.049	0.014
16	Элитарная	BLX	Гауссовская	11.7	0.047	0.013
17	Элитарная	BLX	Равномерная	11.7	0.047	0.012
18	Элитарная	SBX	Равномерная	14.2	0.049	0.014

На рисунке 4.1 представлена столбчатая диаграмма средних рангов конфигураций, а на рисунке 4.2 – тепловая карта рангов конфигураций по входным графам.

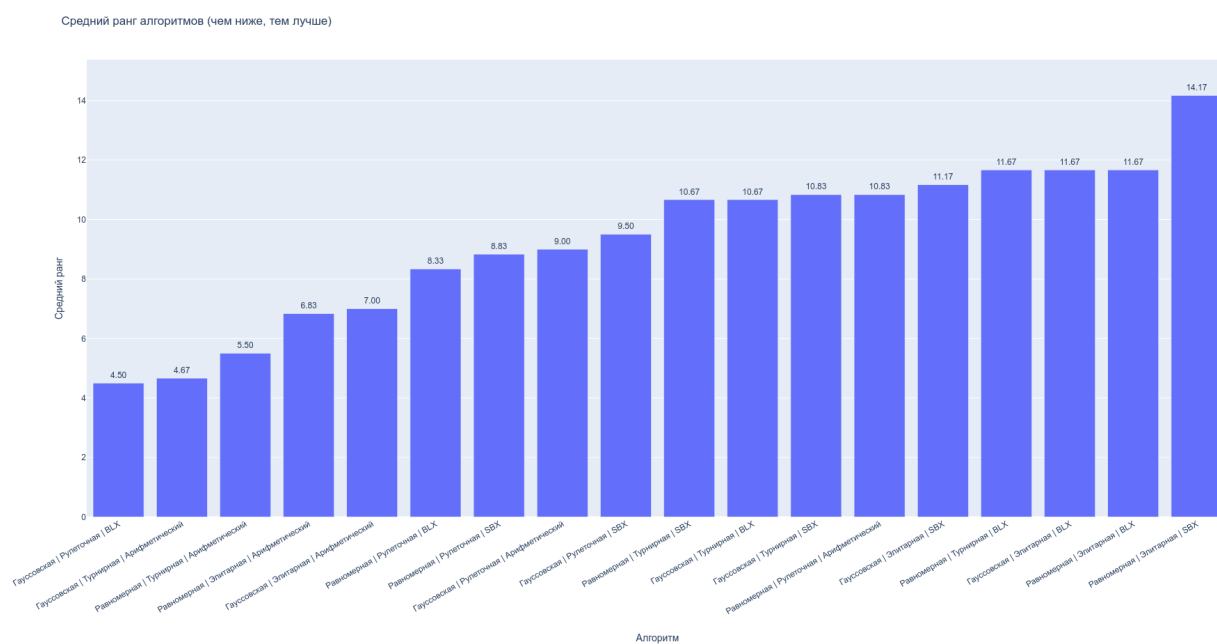


Рисунок 4.1 — Диаграмма средних рангов конфигураций эволюционных операторов

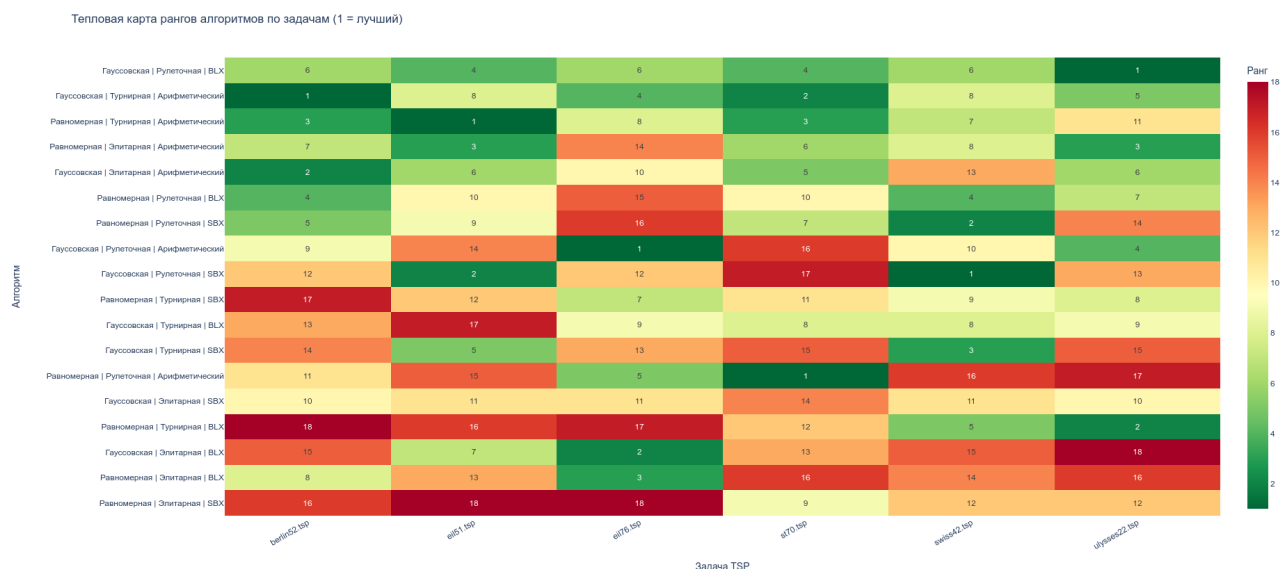


Рисунок 4.2 — Тепловая карта рангов конфигураций эволюционных операторов по входным графам

Таким образом как по среднему рангу, так и по средней относительной ошибке, наиболее удачной конфигурацией является сочетание рулеточной селекции, BLX-скрещивания и гауссовской мутации. Данный вариант будет использован в дальнейших исследованиях как основной.

4.2 Исследование сходимости метода

4.2.1 Цель исследования

Объектом данного исследования является разработанный гетерогенный муравьиный алгоритм с динамическими гетерогенными параметрами на основе генетического алгоритма с оптимальными эволюционными операторами, определёнными в предыдущей секции.

Предметом данного исследования является скорость сходимости алгоритма к оптимальному решению в зависимости от количества поколений и наличия локальной оптимизации.

Цель исследования — оценить влияние количества поколений и локальной оптимизации на сходимость разработанного алгоритма, а также сравнить его сходимость со сходимостью классического муравьиного алгоритма.

4.2.2 Описание исследования

Варьируемыми факторами исследования являются:

- **Количество поколений:** 500, 800, 1000;
- **Тип алгоритма:**
 - классический муравьиный алгоритм;
 - разработанный алгоритм без локальной оптимизации;

- разработанный алгоритм с 2-opt локальной оптимизацией.

Также варьируется входной взвешенный неорграф, описывающий задачу коммивояжёра. Были выбраны 7 тестовых задач из библиотеки TSPLIB: `swiss42.tsp`, `eil51.tsp`, `berlin52.tsp`, `eil76.tsp`, `rd100.tsp`, `eil101.tsp`, `tsp225.tsp`. Прежде всего конфигурации существенно отличаются размерностью графа (от 42 до 225), что позволит оценить эффективность алгоритма при различных размерах входных данных.

Постоянными параметрами и их значениями в данном исследовании являются:

- начальные значения α и β (для классического алгоритма постоянные): 1 и 5 соответственно;
- коэффициент испарения p : 0.5;
- множитель феромона Q : 1;
- количество муравьёв m : n
- начальный уровень феромона τ_0 : $\frac{m}{C_{nn}}$
- период адаптации w : 5;
- количество родителей: $0.4m$;
- эволюционные стратегии: рулеточная селекция, BLX-кроссинговер, гауссовская мутация;

где n – количество вершин в графе, C_{nn} – длина маршрута, построенная жадным алгоритмом.

Для оценки сходимости предлагаются следующие метрики:

- количество итераций до лучшего решения (ИТВ) – номер итерации, на которой алгоритм впервые достигает наилучшего найденного им решения. Меньшее значение ИТВ свидетельствует о более быстрой сходимости;
- длина наилучшего маршрута – длина наилучшего маршрута, найденного алгоритмом за все поколения;
- время затраченное методом.

Алгоритм исследования

- 1) Для каждой тестовой задачи:
 -) загрузить граф из TSP-файла;
 -) вычислить жадный тур для инициализации феромонов.
- 2) Для каждого количества поколений:
 - Для каждого типа алгоритма:
 - 2.1.1. выполнить R независимых запусков;
 - 2.1.2. для каждого запуска зафиксировать:
 - длину найденного маршрута;
 - количество итераций до лучшего решения (ИТВ);
 - время затраченное алгоритмом;
- 3) Для каждого количества поколений и типа алгоритма вычислить средние и стандартные отклонения метрик по 20 запускам.

4.2.3 Результаты исследования

Исследование проводилось на устройстве со следующими характеристиками:

- Процессор: AMD Ryzen 7 5800H (16 ядер, 4.46 ГГц);
- Оперативная память: 16 ГБ;
- Операционная система: openSUSE Tumbleweed x86_64 (Linux 6.19.12-1-default).

Для каждой пары алгоритма и входного графа алгоритм запускался $R = 20$ раз. Полученные характеристики запусков представлены в таблице 4.2, где:

- N_g – количество поколений;
- μ_r – средняя длина полученного маршрута по R запускам;
- σ_r – среднее квадратичное отклонение длины полученного маршрута по R запускам;
- $ITB_{1/2}$ – медиана количества итераций до получения лучшего решения;
- μ_t – среднее время работы алгоритма в миллисекундах.

Таблица 4.2 — Результаты исследования сходимости алгоритмов

Файл	N_g	Алгоритм	μ_r	σ_r	$ITB_{1/2}$	μ_t , мс
berlin52	500	мод.	7679.35	33.54	120.50	4023.05
		мод. + лок.	7542.00	0.00	55.00	4474.95
		мур.	7689.85	39.10	173.50	3705.10
	800	мод.	7665.15	62.90	185.50	6425.40
		мод. + лок.	7542.00	0.00	35.00	7115.70
		мур.	7654.50	40.85	164.00	5860.85
	1000	мод.	7677.95	35.46	229.50	8047.80
		мод. + лок.	7542.00	0.00	80.00	8905.05
		мур.	7643.35	55.09	222.00	7296.10
eil101	500	мод.	703.80	8.53	39.00	35346.20
		мод. + лок.	637.90	2.69	187.50	50538.70
		мур.	691.50	6.19	311.00	36709.70
	800	мод.	700.15	7.58	80.50	53467.15
		мод. + лок.	636.10	2.95	205.00	80859.70
		мур.	690.20	5.49	386.50	57082.30
	1000	мод.	698.75	9.82	196.50	60207.80
		мод. + лок.	636.50	2.06	355.00	96070.85
		мур.	689.40	7.54	306.00	72192.30
eil51	500	мод.	449.65	7.48	63.50	3780.50
		мод. + лок.	428.80	1.47	100.00	4523.50
		мур.	449.15	4.84	77.50	3519.50
	800	мод.	451.70	5.54	86.00	6070.05
		мод. + лок.	429.05	1.23	97.50	7191.95

Таблица 4.2 — Результаты исследования сходимости (продолжение)

Файл	N_g	Алгоритм	μ_r	σ_r	$ITB_{1/2}$	μ_t , мс
	1000	мур.	445.60	4.63	120.00	5580.60
		мод.	448.60	6.77	76.50	7597.35
		мод. + лок.	428.50	1.19	217.50	9018.25
		мур.	445.25	5.40	195.50	6943.40
eil76	500	мод.	568.45	3.95	49.50	12146.80
		мод. + лок.	543.95	2.39	122.50	14738.05
		мур.	564.80	1.74	183.00	11025.75
	800	мод.	567.65	4.37	57.50	19408.70
		мод. + лок.	544.35	3.47	155.00	23688.75
		мур.	564.05	1.88	267.50	17671.55
	1000	мод.	567.25	1.83	91.50	25063.00
		мод. + лок.	544.40	2.16	200.00	29842.45
		мур.	563.35	1.60	384.50	22100.75
rd100	500	мод.	8577.05	84.01	45.50	36749.10
		мод. + лок.	8030.50	35.19	172.50	42909.55
		мур.	8516.20	59.96	37.50	32748.25
	800	мод.	8559.65	63.32	80.00	59605.30
		мод. + лок.	8033.50	32.15	160.00	66773.70
		мур.	8512.40	75.65	114.00	52357.40
	1000	мод.	8541.50	108.99	115.00	75370.50
		мод. + лок.	8021.15	33.95	92.50	86659.00
		мур.	8520.55	51.46	157.50	65199.05
swiss42	500	мод.	1298.05	4.81	24.50	2170.00
		мод. + лок.	1273.00	0.00	32.50	2402.15
		мур.	1298.40	2.68	13.50	1957.15
	800	мод.	1295.40	5.01	34.00	3532.80
		мод. + лок.	1273.00	0.00	20.00	3831.10
		мур.	1297.70	3.20	25.00	3212.60
	1000	мод.	1295.20	5.69	35.00	4566.05
		мод. + лок.	1273.00	0.00	20.00	4818.10
		мур.	1298.60	1.05	24.50	4012.65
tsp225	1000	мод.	4303.80	32.84	45.50	796904.15
		мод. + лок.	3994.10	14.79	212.50	1041099.90
		мур.	4259.60	19.64	235.00	608841.30

На рисунке 4.3 представлена коробочная диаграмма, отображающая распределение ме-

дианы ИТВ полученным алгоритмами в задачах, а на рисунке 4.4 – точечный график зависимости медиан длины маршрута и ИТВ разными алгоритмами по задачам.

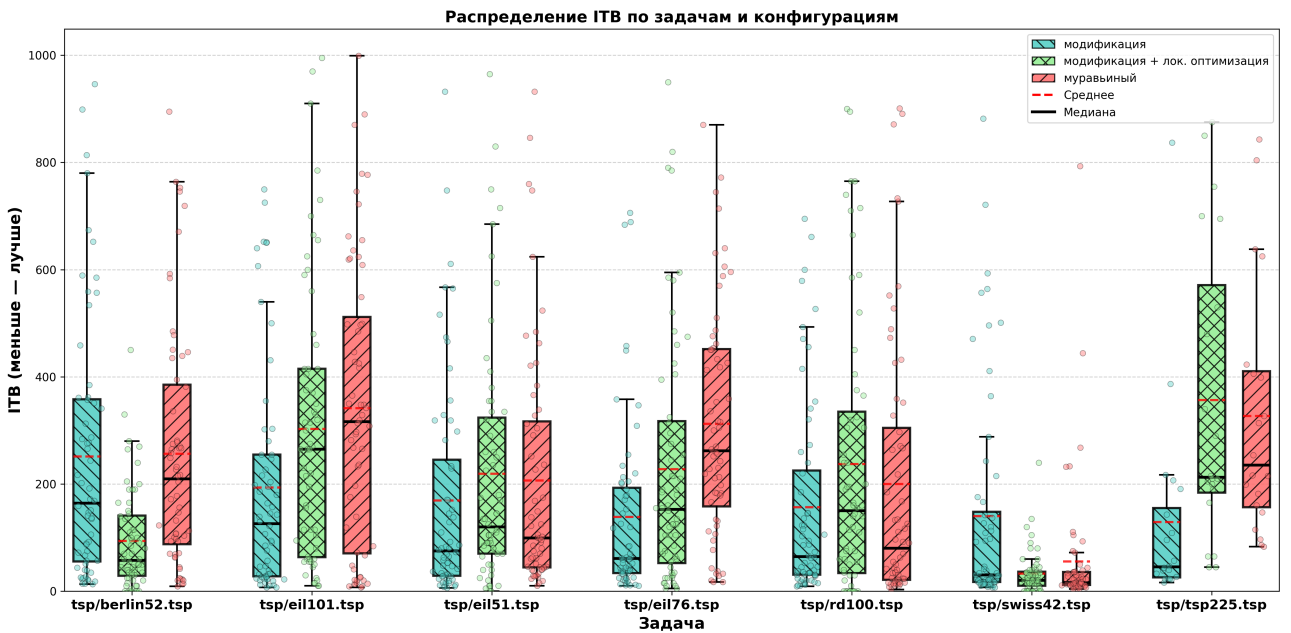


Рисунок 4.3 — Коробчатая диаграмма распределения ИТВ по задачам и алгоритмам

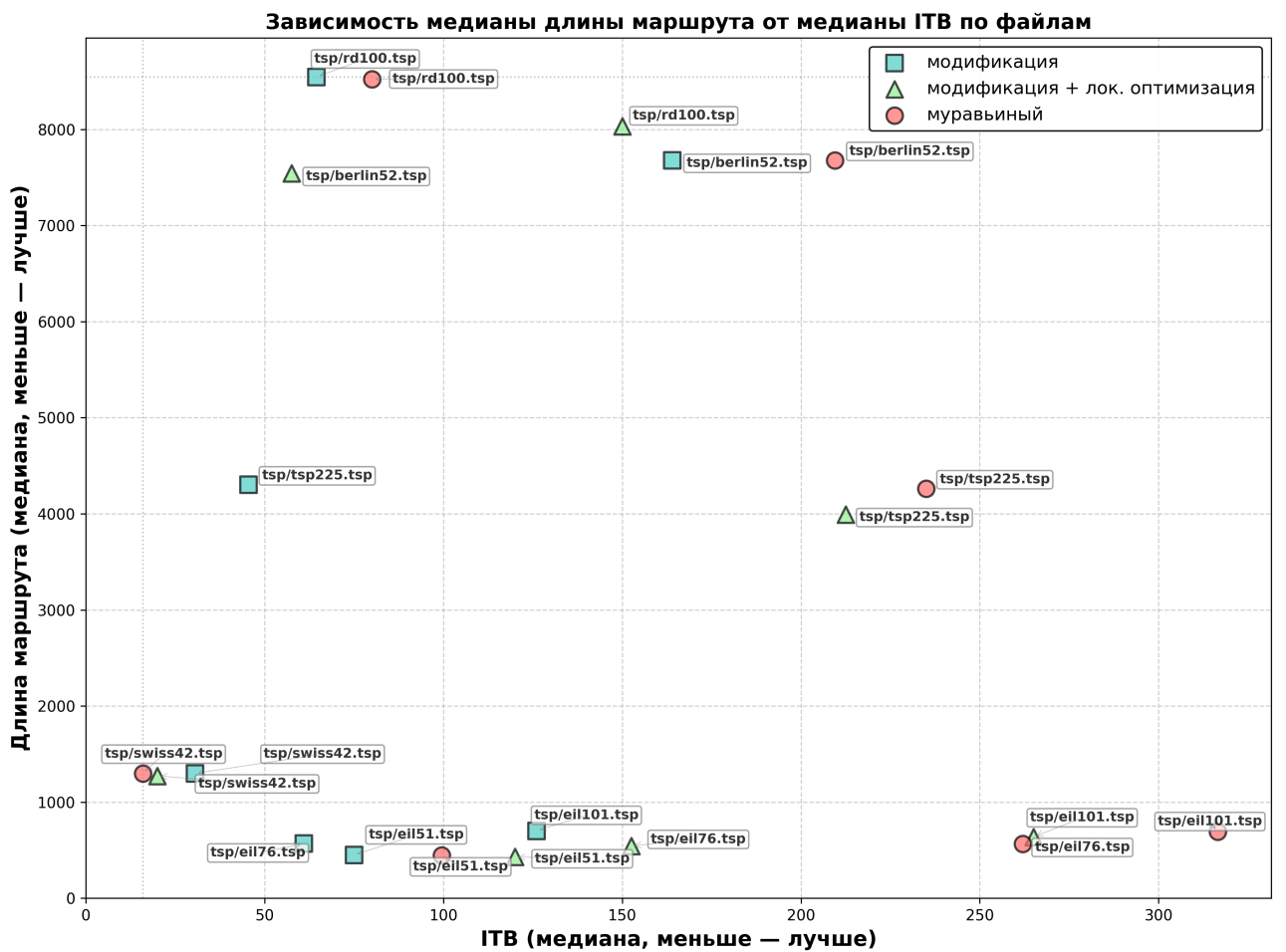


Рисунок 4.4 — Точечный график зависимости ИТВ и длины найденного маршрута по задачам и алгоритмам

Разработанный алгоритм с локальной оптимизацией без локальной оптимизации (мод.) демонстрирует качество, хоть и несколько более худшее, но сопоставимое с классическим муравьиным алгоритмом (разница в пределах 1–2%). Однако достигает этого качества меньшим количеством итераций: на eil76 ускорение по ИТВ составило 3.7 раза (49.5 против 183.0 итераций), на eil101 — до 8 раз (39.0 против 311.0), на tsp225 — 5.2 раза (45.5 против 235.0). С точки зрения быстродействия, модификация просчитывает тоже число поколение на 10% дольше, относительно классического метода.

4.3 Исследование устойчивости метода

4.3.1 Цель исследования

Объектом данного исследования является разработанный гетерогенный муравьиный алгоритм с динамическими гетерогенными параметрами на основе генетического алгоритма с оптимальными эволюционными операторами, определёнными ранее.

Предметом данного исследования является зависимость качества решений, получаемых алгоритмом, от начальных значений параметров α и β , управляющих влиянием феромона и эвристической информации.

Цель исследования – оценить устойчивость разработанного алгоритма к выбору параметров α и β в сравнении с классическим муравьиным алгоритмом.

4.3.2 Описание исследования

Варьируемыми факторами исследования являются:

- начальное значение параметра α : от 0.5 до 4.0 с шагом 0.5;
- начальное значение параметра β : от 1.0 до 7.0 с шагом 1.0;
- тип алгоритма: классический муравьиный алгоритм и разработанная модификация.

В качестве входного графа был выбран файл в формате TSPLIB eil76.tsp, имеющей размерность в 76 узлов, что более существенно оценить зависимость от параметров.

Постоянными параметрами и их значениями в данном исследовании являются:

- коэффициент испарения p : 0.5;
- множитель феромона Q : 1;
- количество муравьёв m : n
- начальный уровень феромона τ_0 : $\frac{m}{C_{nn}}$
- количество поколений: 500;
- период адаптации w : 5;
- количество родителей: $0.4m$;
- эволюционные стратегии: рулеточная селекция, BLX-кроссинговер, гауссовская мутация;

где n – количество вершин в графе, C_{nn} – длина маршрута, построенная жадным алгоритмом.

Для оценки устойчивости метода предлагаются следующие метрики:

Размах r – разность между максимальным и минимальным значениями средней длины маршрута при варьировании параметров α и β :

$$r = \max L(\alpha, \beta) - \min L(\alpha, \beta).$$

Размах позволяет оценить стабильность алгоритма для конкретных значений α и β .

Коэффициент вариации

μ_L – среднее арифметическое длины маршрута по R запускам для заданной комбинации (α, β) :

$$\mu_L(\alpha, \beta) = \frac{1}{R} \sum_{r=1}^R L_r(\alpha, \beta).$$

σ_L – среднее квадратичное отклонение длины маршрута по R запускам для заданной комбинации (α, β) :

$$\sigma_L(\alpha, \beta) = \sqrt{\frac{1}{R-1} \sum_{r=1}^R (L_r - \mu_L)^2}.$$

CV – относительное среднее квадратичное отклонение, отношение стандартного отклонения средних длин маршрута к их среднему значению по всем комбинациям параметров

$$CV = \frac{\sigma_{\mu_L}}{\mu_{\mu_L}}$$

В данном случае предлагается как интегральная оценка стабильности алгоритма по всем парам α и β в совокупности. Чем меньше CV тем менее разбросаны средние значения путей относительно друг друга и соответственно алгоритм более стабильный.

4.3.3 Результаты исследования

Алгоритм исследования

- 1) Загрузить граф `eil76.tsp` и вычислить жадный тур для инициализации феромонов.
- 2) Для каждого значения α :
 -) Для каждого значения β :
 - 2.1.1. Для каждого типа алгоритма:
 - выполнить R независимых запусков;
 - вычислить размах выборки длин полученных маршрутов;
 - вычислить $\mu_L(\alpha, \beta)$ и $\sigma_L(\alpha, \beta)$
- 3) Для каждого типа алгоритма вычислить CV .

Вывод

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Малых А.Е., Давыдова А.А. РАЗВИТИЕ НЕКОТОРЫХ КЛАССИЧЕСКИХ КОМБИНАТОРНЫХ ЗАДАЧ // ВЕСТНИК ПГГПУ. - 2017. - №2. - С. 82-101.
2. Володина Е.В., Студентова Е.А. Практическое применение алгоритма решения задачи коммивояжёра // Инженерный вестник Дона. - 2015. - №2.
3. Воронин Е.В., Бутузова Е.А. ПРИМЕНЕНИЕ ЗАДАЧИ КОММИВОЯЖЁРА // Символ науки. - Уфа: Символ науки, 2020. - С. 62-63.
4. Математический энциклопедический словарь.. - М.: Советская энциклопедия, 1988. - 848 с.
5. Alexander Schrijver. On the history of combinatorial optimization (till 1960) // 2005.
6. CodeNRock // E-CUP 2025: ML Challenge URL: <https://codenrock.com/contests/e-cup-2025> (дата обращения: 09.11.2025).
7. Teng Fei, Xinxin Wu, Liyi Zhang, Yong Zhang, Lei Chen Research on improved ant colony optimization for traveling salesman problem // Mathematical Biosciences and Engineering. - 2022. - №19. - С. 8152-8186.
8. Мудров В.И. Задача о Коммивояжёре. - М.: Знание, 1969. - 64 с.
9. Белоусов А.И., Ткачев С.Б. Дискретная математика: учебник для вузов. - 7-е изд. - М.: Издательство МГТУ им. Н. Э. Баумана, 2021. - 703 с.
10. Marco Dorigo, Vittorio Maniezzo, Alberto Colorni. The Ant System: Optimization by a colony of cooperating agents // IEEE Transactions on Systems, Man, and Cybernetics – Part B. - 1996. - №1. - С. 1-13.
11. Eric Bonabeau, Marco Dorigo, Guy Theraulaz Swarm Intelligence From Natural to Artificial Systems. - Oxford: Oxford University Press, 1999. - 307 с.
12. Thomas Shuzle, Holger H. Hobs Max-Min Ant System // Future generations computer systems. - 2000. - №16. - С. 889-914.
13. Bemd BulInheimer, Richard F. Hartl, Christine Strauss A New Rank Based Version of the Ant System - A Computational Study // Central European Journal of Operation Research. - 1999. - №7. - С. 25-38.
14. Rico Wijaya Dewantoro, Poltak Sihombing, Sutarman The Combination of Ant Colony Optimization (ACO) and Tabu Search (TS) Algorithm to Solve the Traveling Salesman Problem (TSP) // The 3-rd International Conference on Electrical, Telecommunication and Computer Engineeri. - 2019. - С. 160-164.

15. MEIJIAO LIU, YANHUI LI A Slime Mold-Ant Colony Fusion Algorithm for Solving Traveling Salesman Problem // IEEE Access. - 2020. - №8
16. Zainudin Zukhri, Irving Vitra Paputungan A Hybrid Optimization Algorithm Based On Genetic Alhorithm And Ant Colony Optimization // International Journal of Artificial Intelligence & Application. - 2013. - №5. - С. 63-75.
17. Kezong Tang, Xiong-Fei Wei, Yuan-Hao Jiang, Zi-Wei Chen, Lihua Yang An Adaptive Ant Colony Optimization for Solving Large-Scale Traveling Salesman Problem // Mathematics. - 2023. - №11
18. Petr Stodola, Pavel Otrisal, Kamila Hasilova Adaptive Ant Colony Optimization with node clustering applied to the Travelling Salesman Problem // Swarm and Evolutionary Computation. - 2022. - №70
19. Ahamed Fayeez Tuani, Edward Keedwell, Matthew Collett Heterogenous Adaptive Ant Colony Optimization with 3-opt local search for the Travelling Salesman Problem // Applied Soft Computing Journal. - 2020
20. John H. Holland Adaptation in Natural and Artificial Systems . - The MIT Press, 1975. - 232 с.
21. Скобцов Ю. А. Генетические алгоритмы в программной инженерии. - М.: "Инфра-Инженерия 2025. - 144 с.
22. Luu Van Thanh Variants of 2-opt Approach for the Generalized Traveling Salesman Problem // International Journal of Science and Research (IJSR). - 2018. - №28. - С. 1554-1564.
23. Local Search for Traveling Salesman Problem // DM865 – Spring 2020, Heuristics and Approximation Algorithms URL: <https://dm865.github.io/assets/dm865-tsp-ls-handout.pdf> (дата обращения: 13.03.2026).
24. Marco Dorigo, Thomas Stutzle Ant colony optimization. - Massachusetts: The MIT Press, 2004. - 304 с.
25. Кормен, Томас Х., Лейзерсон, Чарльз И., Ривест, Рональд Л., Штайн, Клиффорд Алгоритмы: построение и анализ. - 2-е изд. - М.: Издательский дом Вильямс, 2013. - 1296 с.
26. Никлаус Вирт Алгоритмы и струкуры данных. - 2-е изд. - М.: ДМК Пресс, 2016. - 272 с.
27. Гамма Э., Хелм Р., Джонсон Р., Влиссидес Дж. Паттерны объектно-ориентированного проектирования. - СПб: Питер, 2020. - 448 с.
28. TSPLIB 95 // Discrete and Combinatorial Optimization URL: <https://comopt.ifl.uni-heidelberg.de/software/TSPLIB95/tsp95.pdf> (дата обращения: 13.05.2026).
29. The Go programming language // URL: <https://go.dev/> (дата обращения: 13.05.2026).

30. Gin Web Framework // Gin Web Framework URL: <https://gin-gonic.com/> (дата обращения: 13.05.2026).
31. TypeScript is JavaScript with syntax for types // TypeScript URL: <https://www.typescriptlang.org/> (дата обращения: 13.05.2026).
32. Interactive data visualization // Plotly URL: <https://plotly.com/> (дата обращения: 13.05.2026).

Приложение Б

Таблица 4.3 — Результаты для задачи berlin52.tsp. Оптимальное решение 7542.00

№	Селекция	Кроссинговер	Мутация	Среднее	СКО	RSE (%)
1	Турнирная	Арифметический	Гауссовская	7702.60	99.69	1.29
2	Элитарная	Арифметический	Гауссовская	7710.55	101.45	1.32
3	Турнирная	Арифметический	Равномерная	7713.65	80.61	1.05
4	Рулеточная	BLX	Равномерная	7716.30	119.48	1.55
5	Рулеточная	SBX	Равномерная	7723.05	119.87	1.55
6	Рулеточная	BLX	Гауссовская	7723.65	115.91	1.50
7	Элитарная	Арифметический	Равномерная	7728.20	92.56	1.20
8	Элитарная	BLX	Равномерная	7730.95	91.62	1.19
9	Рулеточная	Арифметический	Гауссовская	7737.50	124.63	1.61
10	Элитарная	SBX	Гауссовская	7737.70	88.18	1.14
11	Рулеточная	Арифметический	Равномерная	7738.15	118.89	1.54
12	Рулеточная	SBX	Гауссовская	7742.40	112.46	1.45
13	Турнирная	BLX	Гауссовская	7744.30	112.20	1.45
14	Турнирная	SBX	Гауссовская	7749.40	106.03	1.37
15	Элитарная	BLX	Гауссовская	7760.25	95.39	1.23
16	Элитарная	SBX	Равномерная	7772.45	123.11	1.58
17	Турнирная	SBX	Равномерная	7785.40	123.67	1.59
18	Турнирная	BLX	Равномерная	7819.20	105.16	1.34

Таблица 4.4 — Результаты для задачи eil51.tsp. Оптимальное решение 426.00

№	Селекция	Кроссинговер	Мутация	Среднее	СКО	RSE (%)
1	Турнирная	Арифметический	Равномерная	451.25	8.49	1.88
2	Рулеточная	SBX	Гауссовская	451.50	6.24	1.38
3	Элитарная	Арифметический	Равномерная	452.30	6.96	1.54
4	Рулеточная	BLX	Гауссовская	452.40	8.15	1.80
5	Турнирная	SBX	Гауссовская	453.65	10.70	2.36
6	Элитарная	Арифметический	Гауссовская	453.85	8.74	1.93
7	Элитарная	BLX	Гауссовская	454.00	8.45	1.86
8	Турнирная	Арифметический	Гауссовская	454.15	9.15	2.01
9	Рулеточная	SBX	Равномерная	454.65	9.27	2.04
10	Рулеточная	BLX	Равномерная	454.80	9.17	2.02
11	Элитарная	SBX	Гауссовская	455.15	9.66	2.12

Таблица 4.4 — Результаты для задачи eil51.tsp (продолжение)

№	Селекция	Кроссинговер	Мутация	Среднее	СКО	RSE (%)
12	Турнирная	SBX	Равномерная	455.95	9.56	2.10
13	Элитарная	BLX	Равномерная	456.15	8.33	1.83
14	Рулеточная	Арифметический	Гауссовская	456.30	8.25	1.81
15	Рулеточная	Арифметический	Равномерная	457.05	11.41	2.50
16	Турнирная	BLX	Равномерная	457.20	9.29	2.03
17	Турнирная	BLX	Гауссовская	458.45	8.04	1.75
18	Элитарная	SBX	Равномерная	459.50	9.80	2.13

Таблица 4.5 — Результаты для задачи eil76.tsp. Оптимальное решение 538.00

№	Селекция	Кроссинговер	Мутация	Среднее	СКО	RSE (%)
1	Рулеточная	Арифметический	Гауссовская	570.75	4.49	0.79
2	Элитарная	BLX	Гауссовская	571.15	7.15	1.25
3	Элитарная	BLX	Равномерная	571.20	5.09	0.89
4	Турнирная	Арифметический	Гауссовская	571.30	5.45	0.95
5	Рулеточная	Арифметический	Равномерная	571.40	5.58	0.98
6	Рулеточная	BLX	Гауссовская	571.80	3.81	0.67
7	Турнирная	SBX	Равномерная	571.90	5.11	0.89
8	Турнирная	Арифметический	Равномерная	572.20	4.46	0.78
9	Турнирная	BLX	Гауссовская	572.25	5.32	0.93
10	Элитарная	Арифметический	Гауссовская	572.70	4.65	0.81
11	Элитарная	SBX	Гауссовская	572.95	4.19	0.73
12	Рулеточная	SBX	Гауссовская	573.15	6.65	1.16
13	Турнирная	SBX	Гауссовская	573.25	5.29	0.92
14	Элитарная	Арифметический	Равномерная	573.95	5.28	0.92
15	Рулеточная	BLX	Равномерная	574.05	5.69	0.99
16	Рулеточная	SBX	Равномерная	574.20	7.32	1.27
17	Турнирная	BLX	Равномерная	575.00	7.66	1.33
18	Элитарная	SBX	Равномерная	575.25	4.89	0.85

Таблица 4.6 — Результаты для задачи st70.tsp. Оптимальное решение 675.00

№	Селекция	Кроссинговер	Мутация	Среднее	СКО	RSE (%)
1	Рулеточная	Арифметический	Равномерная	731.15	11.12	1.52
2	Турнирная	Арифметический	Гауссовская	731.25	10.99	1.50
3	Турнирная	Арифметический	Равномерная	731.30	11.77	1.61

Таблица 4.6 — Результаты для задачи st70.tsp (продолжение)

№	Селекция	Кроссинговер	Мутация	Среднее	СКО	RSE (%)
4	Рулеточная	BLX	Гауссовская	731.35	6.49	0.89
5	Элитарная	Арифметический	Гауссовская	731.50	10.87	1.49
6	Элитарная	Арифметический	Равномерная	731.75	7.11	0.97
7	Рулеточная	SBX	Равномерная	732.95	9.27	1.26
8	Турнирная	BLX	Гауссовская	733.35	10.13	1.38
9	Элитарная	SBX	Равномерная	733.45	13.42	1.83
10	Рулеточная	BLX	Равномерная	733.85	13.72	1.87
11	Турнирная	SBX	Равномерная	735.30	9.47	1.29
12	Турнирная	BLX	Равномерная	735.40	14.03	1.91
13	Элитарная	BLX	Гауссовская	735.90	10.92	1.48
14	Элитарная	SBX	Гауссовская	736.50	10.54	1.43
15	Турнирная	SBX	Гауссовская	736.90	12.42	1.69
16	Рулеточная	Арифметический	Гауссовская	737.25	11.59	1.57
17	Элитарная	BLX	Равномерная	737.25	7.52	1.02
18	Рулеточная	SBX	Гауссовская	737.45	9.74	1.32

Таблица 4.7 — Результаты для задачи swiss42.tsp. Оптимальное решение 1273.00

№	Селекция	Кроссинговер	Мутация	Среднее	СКО	RSE (%)
1	Рулеточная	SBX	Гауссовская	1294.45	10.23	0.79
2	Рулеточная	SBX	Равномерная	1295.30	10.41	0.80
3	Турнирная	SBX	Гауссовская	1295.55	7.36	0.57
4	Рулеточная	BLX	Равномерная	1296.95	8.97	0.69
5	Турнирная	BLX	Равномерная	1297.00	7.86	0.61
6	Рулеточная	BLX	Гауссовская	1297.05	6.49	0.50
7	Турнирная	Арифметический	Равномерная	1298.15	8.94	0.69
8	Турнирная	BLX	Гауссовская	1299.05	12.37	0.95
9	Турнирная	Арифметический	Гауссовская	1299.05	4.52	0.35
10	Элитарная	Арифметический	Равномерная	1299.05	9.92	0.76
11	Турнирная	SBX	Равномерная	1299.70	9.18	0.71
12	Рулеточная	Арифметический	Гауссовская	1299.75	10.32	0.79
13	Элитарная	SBX	Гауссовская	1300.05	15.65	1.20
14	Элитарная	SBX	Равномерная	1301.05	11.04	0.85
15	Элитарная	Арифметический	Гауссовская	1301.60	9.16	0.70
16	Элитарная	BLX	Равномерная	1304.55	14.49	1.11

Таблица 4.7 — Результаты для задачи swiss42.tsp (продолжение)

№	Селекция	Кроссинговер	Мутация	Среднее	СКО	RSE (%)
17	Элитарная	BLX	Гауссовская	1305.40	11.32	0.87
18	Рулеточная	Арифметический	Равномерная	1309.65	13.81	1.05

Таблица 4.8 — Результаты для задачи ulysses22.tsp. Оптимальное решение 6981.33

№	Селекция	Кроссинговер	Мутация	Среднее	СКО	RSE (%)
1	Рулеточная	BLX	Гауссовская	7005.95	38.25	0.55
2	Турнирная	BLX	Равномерная	7011.61	40.65	0.58
3	Элитарная	Арифметический	Равномерная	7015.87	39.23	0.56
4	Рулеточная	Арифметический	Гауссовская	7016.63	43.12	0.61
5	Турнирная	Арифметический	Гауссовская	7017.27	56.13	0.80
6	Элитарная	Арифметический	Гауссовская	7021.12	40.80	0.58
7	Рулеточная	BLX	Равномерная	7024.42	51.05	0.73
8	Турнирная	SBX	Равномерная	7025.30	66.45	0.95
9	Турнирная	BLX	Гауссовская	7025.36	53.95	0.77
10	Элитарная	SBX	Гауссовская	7027.46	65.68	0.93
11	Турнирная	Арифметический	Равномерная	7029.80	44.64	0.63
12	Элитарная	SBX	Равномерная	7032.00	65.38	0.93
13	Рулеточная	SBX	Гауссовская	7035.71	57.29	0.81
14	Рулеточная	SBX	Равномерная	7038.08	49.83	0.71
15	Турнирная	SBX	Гауссовская	7040.33	52.23	0.74
16	Элитарная	BLX	Равномерная	7048.69	55.91	0.79
17	Рулеточная	Арифметический	Равномерная	7050.75	58.22	0.83
18	Элитарная	BLX	Гауссовская	7055.78	61.26	0.87